

Análise e Projeto de Sistemas

DA IDEIA AO SISTEMA EM FUNCIONAMENTO



Diagrama de Caso de Uso



Diagrama de Classes (UML)



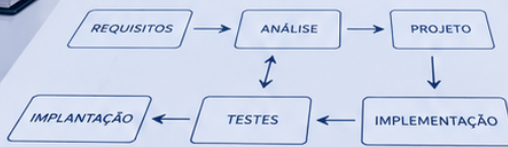
*Sistemas melhores
são construídos
por pessoas.*

- PESSOAS**
Papéis e responsabilidades
- PROCESSOS**
Metodologias e boas práticas
- TECNOLOGIA**
Modelagem e desenvolvimento
- RESULTADOS**
Soluções para o mundo real

ANALISAR
MODELAR
PROJETAR
DESENVOLVER
EM EQUIPE
EVOLUIR

BOAS
IDEIAS
GERAM
GRANDES
SISTEMAS

UML
ENGENHARIA DE SOFTWARE
MODELAGEM DE SISTEMAS
PROJETO ORIENTADO A OBJETOS



PLANEJAR
MODELAR
DESENVOLVER
EVOLUIR

SISTEMAS
PESSOAS
OPORTUNIDADES

Sumário

1	Fundamentos de Análise de Sistemas	6
1.1	Análise de Sistemas	6
1.2	Modelagem de Sistemas	7
1.3	Paradigma da Orientação a Objetos	8
1.4	Classes e Objetos	8
1.4.1	Classe	9
1.4.2	Objeto	9
1.5	Mensagens entre Objetos	9
1.6	Diferentes Visões de um Sistema	10
1.7	Exemplo Integrador	11
1.8	Resumo do Capítulo	12
1.9	Fixando os Conceitos	12
1.10	Atividade Prática	12
2	Processo de Desenvolvimento de Software	15
2.1	O que é um Processo de Desenvolvimento de Software?	16
2.2	Levantamento de Requisitos	16
2.2.1	Requisitos Funcionais e Não Funcionais	17
2.3	Análise	18
2.4	Projeto	18
2.5	Análise ou Projeto?	19
2.6	Implementação	20
2.7	Testes	20
2.7.1	Tipos de Testes	21
2.8	Implantação	21
2.9	Visão Geral do Processo	22
2.10	Exemplo Integrador: Biblioteca Escolar	22
2.11	Importante: o Processo Nem Sempre é Linear	22
2.12	Resumo do Capítulo	23
2.13	Fixando os Conceitos	23
2.14	Atividade Prática	24
3	O Componente Humano no Desenvolvimento de Software	25
3.1	Software é Construído por Pessoas	26
3.2	Gerente de Projetos	27
3.2.1	Exemplo	27
3.3	Analista de Sistemas	28

3.3.1	Exemplo	28
3.4	Projetista de Sistemas	29
3.4.1	Exemplo	29
3.5	Arquiteto de Software	30
3.6	Programador	30
3.6.1	Exemplo	31
3.7	Especialista do Domínio	31
3.7.1	Analista e Especialista do Domínio	32
3.8	Avaliador de Qualidade	32
3.9	Os Profissionais Trabalham em Conjunto	33
3.10	Quem Faz o Quê?	33
3.11	Exemplo Integrador: Biblioteca Escolar	33
3.12	Importante: Papéis não são Pessoas	34
3.13	Resumo do Capítulo	34
3.14	Fixando os Conceitos	35
3.15	Atividade Prática	35
4	Modelos de Ciclo de Vida de Software	36
4.1	O que é um Modelo de Ciclo de Vida?	37
4.2	Modelos Prescritivos de Processo	37
4.3	Modelo Cascata	38
4.3.1	Exemplo	38
4.3.2	Vantagens	39
4.3.3	Limitações	39
4.4	Modelo Incremental	39
4.5	RAD – Rapid Application Development	40
4.6	Prototipagem	40
4.6.1	Exemplo	41
4.7	Modelo Espiral	41
4.7.1	O que é um risco?	42
4.8	Processo Unificado	42
4.8.1	Concepção	43
4.8.2	Elaboração	43
4.8.3	Construção	43
4.8.4	Transição	43
4.9	Desenvolvimento Ágil	44
4.10	Extreme Programming – XP	44
4.10.1	Programação em Pares	45
4.10.2	Refatoração	45
4.10.3	Ciclo Simplificado de XP	45
4.11	Scrum	46
4.11.1	Scrum Team	46
4.11.2	Product Owner	46
4.11.3	Scrum Master	46
4.11.4	Developers	46
4.11.5	Artefatos	46
4.11.6	Eventos	47

4.11.7 Fluxo Simplificado	47
4.12 XP e Scrum são a Mesma Coisa?	47
4.13 Comparando os Modelos	48
4.14 Exemplo Integrador: Biblioteca Escolar	48
4.15 Qual é o Melhor Modelo?	49
4.16 Resumo do Capítulo	49
4.17 Fixando os Conceitos	49
4.18 Atividade Prática	50
5 Introdução à UML	51
5.1 O que é UML?	51
5.2 A UML não é uma Linguagem de Programação	52
5.3 A UML também não é um Processo de Desenvolvimento	53
5.4 Por que Utilizar UML?	53
5.4.1 Uma Analogia	54
5.5 Elementos Básicos da UML	54
5.5.1 Ator	55
5.5.2 Caso de Uso	55
5.5.3 Classe	55
5.5.4 Relacionamentos	55
5.5.5 Notas	55
5.6 O que é um Diagrama?	56
5.7 Tipos de Diagramas UML	56
5.8 Diagramas Estruturais	57
5.9 Diagramas Comportamentais	57
5.10 Diagramas de Interação	57
5.11 Precisamos Utilizar Todos os Diagramas?	58
5.12 Visões de um Sistema	58
5.12.1 Visão de Casos de Uso	58
5.12.2 Visão de Projeto	58
5.12.3 Visão de Processo	59
5.12.4 Visão de Implementação	59
5.12.5 Visão de Implantação	59
5.12.6 Representação das Visões	59
5.13 UML e Análise de Sistemas	60
5.14 UML e Projeto de Sistemas	60
5.15 Da Análise ao Projeto	61
5.16 Exemplo Integrador: Biblioteca Escolar	61
5.16.1 Funcionalidades	61
5.16.2 Estrutura	61
5.16.3 Interação	62
5.16.4 Atividades	62
5.16.5 Implantação	62
5.17 Resumo dos Principais Diagramas	62
5.18 Resumo do Capítulo	63
5.19 Fixando os Conceitos	63
5.20 Atividade Prática	64

6	Modelagem de Casos de Uso	65
6.1	Modelo de Casos de Uso	66
6.1.1	Parte Gráfica e Parte Textual	66
6.2	Caso de Uso	67
6.2.1	Um Caso de Uso Representa um Objetivo	67
6.2.2	Como Nomear um Caso de Uso	67
6.3	Atores	68
6.3.1	Representação de um Ator	68
6.3.2	Ator é um Papel	68
6.4	Atores Primários e Secundários	69
6.4.1	Ator Primário	69
6.4.2	Ator Secundário	69
6.5	Relacionamentos	69
6.6	Relacionamento de Comunicação	70
6.7	Relacionamento de Inclusão	70
6.8	Relacionamento de Extensão	70
6.9	Include ou Extend?	71
6.10	Generalização	71
6.10.1	Generalização de Atores	71
6.11	Diagrama de Casos de Uso	71
6.12	Fronteira do Sistema	72
6.13	Identificação dos Elementos do MCU	72
6.14	Identificação dos Atores	73
6.14.1	Evite Confundir Ator com Parte do Sistema	74
6.15	Identificação dos Casos de Uso	74
6.15.1	Casos de Uso Primários	74
6.15.2	Casos de Uso Secundários	75
6.16	Construção do Modelo de Casos de Uso	75
6.17	Construção do Diagrama de Casos de Uso	75
6.18	Documentação dos Atores	76
6.19	Documentação dos Casos de Uso	76
6.20	Fluxo Principal	77
6.21	Fluxos Alternativos	77
6.22	Fluxos de Exceção	77
6.23	Pré-condições e Pós-condições	77
6.23.1	Precondição	77
6.23.2	Pós-condição	78
6.24	Exemplo Completo de Documentação	78
6.25	Evite Detalhes de Interface na Análise	78
6.26	Documentação Suplementar ao MCU	79
6.27	Regras de Negócio	79
6.28	Requisitos de Desempenho	80
6.29	Requisitos de Interface Gráfica	80
6.30	MCU em um Processo de Desenvolvimento Iterativo	81
6.31	O MCU nas Atividades de Análise e Projeto	81
6.32	O MCU nas Demais Atividades do Desenvolvimento	82
6.32.1	Planejamento	82

6.32.2	Modelagem de Classes	82
6.32.3	Modelagem de Interações	83
6.32.4	Projeto de Interface	83
6.32.5	Testes	83
6.33	Exemplo Integrador: Sistema de Biblioteca	83
6.33.1	Atores	83
6.33.2	Casos de Uso	83
6.33.3	Regras de Negócio	84
6.33.4	Modelo Simplificado	84
6.34	Erros Comuns na Modelagem de Casos de Uso	84
6.34.1	Representar pequenas ações como casos de uso	84
6.34.2	Colocar elementos internos como atores	85
6.34.3	Confundir MCU com Fluxograma	85
6.34.4	Utilizar relacionamentos em excesso	85
6.34.5	Desenhar e não documentar	85
6.35	Resumo do Capítulo	85
6.36	Fixando os Conceitos	85
6.37	Atividade Prática	86

Capítulo 1

Fundamentos de Análise de Sistemas

Os sistemas computacionais fazem parte de diferentes atividades do nosso cotidiano. Aplicativos de mensagens, sistemas escolares, lojas virtuais, aplicativos bancários e plataformas de jogos são exemplos de sistemas desenvolvidos para atender determinadas necessidades.

Antes de iniciar a programação de um sistema, é importante compreender o problema que deverá ser resolvido, as necessidades dos usuários e as principais características da solução. Esse trabalho faz parte da **análise de sistemas**.

Neste capítulo, serão apresentados conceitos básicos que ajudam a compreender como um sistema pode ser analisado e representado antes de sua implementação.

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- compreender a importância da análise e da modelagem de sistemas;
- reconhecer os conceitos básicos da orientação a objetos;
- diferenciar classes e objetos;
- compreender como os objetos podem trocar mensagens;
- entender o conceito de modelo de um sistema;
- identificar diferentes visões utilizadas para representar um sistema.

1.1 Análise de Sistemas

A análise de sistemas procura compreender um problema antes da construção da solução computacional. Durante essa etapa, são estudadas as necessidades dos usuários, as informações que serão utilizadas e as funcionalidades que o sistema deverá oferecer.

Por exemplo, imagine que uma escola deseja desenvolver um sistema para controlar empréstimos de livros da biblioteca. Antes de programar, algumas perguntas precisam ser respondidas:

- Quem utilizará o sistema?
- Quais informações deverão ser armazenadas?

- Como será realizado o empréstimo de um livro?
- Como será registrada a devolução?
- O que acontece quando um livro está indisponível?

Essas perguntas ajudam a compreender o problema e a definir o que o sistema deverá fazer.

Ideia principal

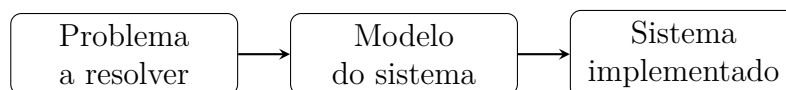
Antes de programar, é necessário compreender o problema que será resolvido.

1.2 Modelagem de Sistemas

Depois de compreender o problema, podemos criar representações simplificadas do sistema. Essas representações são chamadas de **modelos**.

Um **modelo** apresenta apenas as características consideradas importantes para determinado objetivo. É uma representação simplificada de algo que desejamos compreender ou construir.

Podemos comparar um modelo de software com a planta de uma casa. Antes da construção, arquitetos e engenheiros produzem desenhos que representam diferentes partes da construção. Esses desenhos ajudam a compreender como a casa será organizada antes de ela ser construída. No desenvolvimento de software ocorre algo semelhante.



A modelagem facilita a compreensão de sistemas mais complexos e permite que diferentes pessoas envolvidas no desenvolvimento compartilhem uma representação comum da solução. Um modelo pode utilizar:

- desenhos;
- diagramas;
- textos;
- tabelas;
- outras formas de representação.

Na análise e no projeto de sistemas, utilizaremos principalmente diagramas da **UML** – **Linguagem de Modelagem Unificada**, que será estudada com mais detalhes nos próximos capítulos.

1.3 Paradigma da Orientação a Objetos

Existem diferentes maneiras de pensar sobre a construção de programas. Uma delas é o **paradigma da orientação a objetos**. Nesse paradigma, um sistema é compreendido como um conjunto de objetos que possuem responsabilidades e que podem colaborar entre si. Podemos observar objetos no próprio mundo ao nosso redor:

- aluno;
- professor;
- livro;
- computador;
- veículo;
- produto.

Cada objeto possui características e pode realizar determinadas ações. Por exemplo, um objeto que representa um **livro** pode possuir características como:

- título;
- autor;
- ano;
- situação do empréstimo.

Também pode participar de ações como:

- ser emprestado;
- ser devolvido;
- ser reservado.

A orientação a objetos procura representar elementos importantes do problema utilizando objetos computacionais.



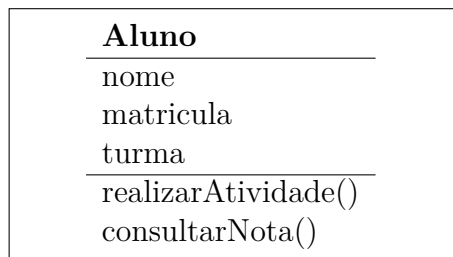
Nos sistemas orientados a objetos, esses elementos colaboram para que as funcionalidades do software sejam realizadas.

1.4 Classes e Objetos

Dois conceitos fundamentais da orientação a objetos são **classe** e **objeto**.

1.4.1 Classe

Uma classe pode ser entendida como uma descrição geral de um conjunto de objetos semelhantes. Ela define quais características e comportamentos os objetos daquele tipo podem possuir. Por exemplo, podemos criar uma classe chamada **Aluno**.



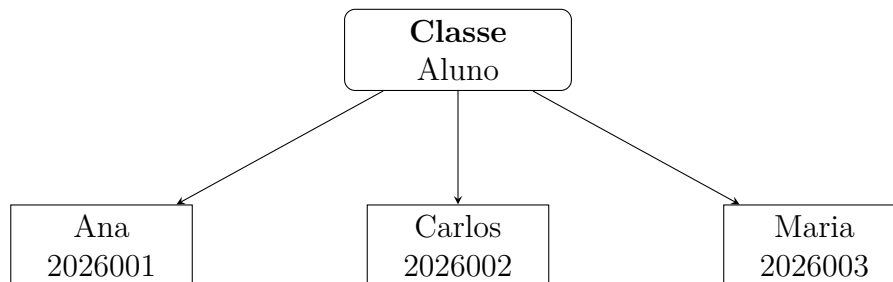
Nesse exemplo, a classe **Aluno** define características que podem ser utilizadas para representar os alunos de uma escola.

1.4.2 Objeto

Um objeto é uma ocorrência concreta de uma classe. Considere os seguintes alunos:

Nome	Matrícula	Turma
Ana	2026001	Informática 1A
Carlos	2026002	Informática 1A
Maria	2026003	Informática 1B

Todos podem ser objetos da classe **Aluno**. A classe define a estrutura geral, enquanto cada objeto possui seus próprios valores.



Assim:

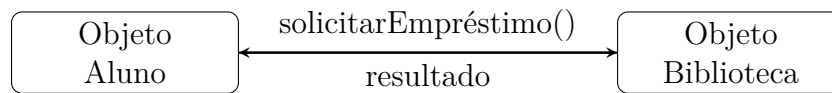
Classe = definição geral ou modelo.

Objeto = elemento específico criado a partir de uma classe.

1.5 Mensagens entre Objetos

Um sistema orientado a objetos normalmente possui vários objetos trabalhando em conjunto. Para realizar determinada tarefa, um objeto pode solicitar que outro objeto execute uma ação. Essa solicitação pode ser compreendida como uma **mensagem**.

Imagine novamente o sistema da biblioteca. Um objeto da classe **Aluno** solicita o empréstimo de um livro. O sistema precisa verificar se o livro está disponível. Podemos representar essa interação de maneira simplificada:



A mensagem indica que um objeto está solicitando algum serviço ou operação a outro objeto. No exemplo:

1. o aluno solicita um empréstimo;
2. o sistema verifica a disponibilidade do livro;
3. uma resposta é produzida;
4. o aluno recebe o resultado da solicitação.

A troca de mensagens permite que diferentes objetos colaborem para executar as funcionalidades do sistema.

Em sistemas orientados a objetos, os objetos podem colaborar por meio da troca de mensagens.

1.6 Diferentes Visões de um Sistema

Um sistema pode ser analisado a partir de diferentes pontos de vista. Imagine um automóvel. Um mecânico pode observar principalmente o motor. Um eletricista pode analisar o sistema elétrico. Um motorista observa os controles utilizados durante a condução. O automóvel é o mesmo, mas cada pessoa observa aspectos diferentes.

Com sistemas de software acontece algo semelhante. A modelagem orientada a objetos pode representar um sistema utilizando diferentes **visões**. Entre as principais estão:

Visão de Casos de Uso: mostra o sistema do ponto de vista de quem o utiliza e das funcionalidades oferecidas.

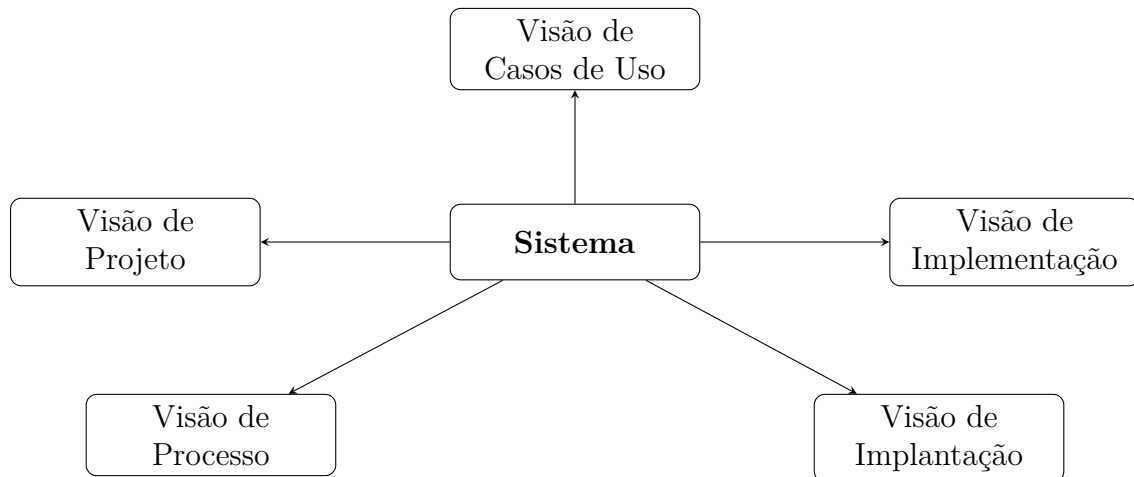
Visão de Projeto: apresenta aspectos estruturais e comportamentais utilizados na construção do sistema.

Visão de Implementação: representa a organização dos componentes e módulos utilizados para implementar o software.

Visão de Implantação: representa onde o software será executado e como seus componentes podem ser distribuídos nos equipamentos.

Visão de Processo: apresenta aspectos relacionados à execução dos processos do sistema, incluindo concorrência, sincronização e desempenho.

A figura a seguir apresenta essas visões de maneira simplificada.



Não é necessário utilizar todas as visões em todos os projetos. A escolha depende das características e da complexidade do sistema que está sendo desenvolvido.

1.7 Exemplo Integrador

Considere um sistema escolar utilizado para registrar atividades dos alunos. Podemos identificar inicialmente alguns elementos:

- Aluno;
- Professor;
- Turma;
- Atividade;
- Nota.

Na orientação a objetos, esses elementos podem ser representados por classes. Um aluno específico, chamado João, pode ser representado como um objeto da classe **Aluno**. Uma atividade específica de programação pode ser um objeto da classe **Atividade**. Quando João envia sua atividade, diferentes objetos do sistema podem interagir:



Esse exemplo mostra como os conceitos apresentados neste capítulo estão relacionados:

Problema → Modelagem → Classes e Objetos → Interações

1.8 Resumo do Capítulo

Neste capítulo, vimos que a análise de sistemas ajuda a compreender o problema antes da implementação do software.

A modelagem permite representar de forma simplificada os principais elementos e comportamentos do sistema.

No paradigma da orientação a objetos, um sistema é compreendido como um conjunto de objetos que possuem características, responsabilidades e podem colaborar entre si.

Uma **classe** descreve características e comportamentos comuns, enquanto um **objeto** representa uma ocorrência específica dessa classe.

Os objetos podem colaborar por meio da troca de **mensagens**.

Também vimos que um mesmo sistema pode ser representado utilizando diferentes visões, cada uma destacando determinados aspectos da solução.

1.9 Fixando os Conceitos

1. O que significa analisar um sistema antes de programá-lo?
2. Explique, com suas palavras, o que é um modelo.
3. Qual é a diferença entre uma classe e um objeto?
4. Considere a classe **Computador**. Cite quatro possíveis características dessa classe.
5. Cite três possíveis objetos da classe **Aluno**.
6. O que significa dizer que dois objetos trocam mensagens?
7. Por que um sistema pode possuir diferentes visões?
8. Considere um sistema de vendas. Identifique pelo menos quatro possíveis classes.

1.10 Atividade Prática

Escolha um dos seguintes sistemas:

- biblioteca escolar;
- cantina escolar;
- sistema de matrícula;
- loja de informática;
- sistema de empréstimo de equipamentos.

Para o sistema escolhido:

1. descreva brevemente o problema que o sistema deverá resolver;

2. identifique pelo menos quatro possíveis classes;
3. apresente dois exemplos de objetos;
4. descreva uma possível troca de mensagens entre dois objetos;
5. explique qual seria a principal função do sistema.

Solução da Atividade Prática

Sistema escolhido: Biblioteca Escolar

1. Problema que o sistema deverá resolver

O sistema deverá auxiliar no controle dos livros disponíveis na biblioteca, permitindo registrar alunos, livros, empréstimos e devoluções.

O objetivo é facilitar a organização do acervo e permitir que a escola saiba quais livros estão disponíveis, quais estão emprestados e quais alunos estão responsáveis pelos empréstimos.

2. Possíveis classes do sistema

Algumas classes que podem ser identificadas são:

- Aluno;
- Livro;
- Emprestimo;
- Bibliotecario.

Cada classe representa um tipo de elemento importante para o funcionamento do sistema.

3. Exemplos de objetos

Um objeto é uma ocorrência específica de uma classe.

Por exemplo:

- João Silva pode ser um objeto da classe **Aluno**;
- o livro “Introdução à Informática” pode ser um objeto da classe **Livro**.

Assim, podemos representar:

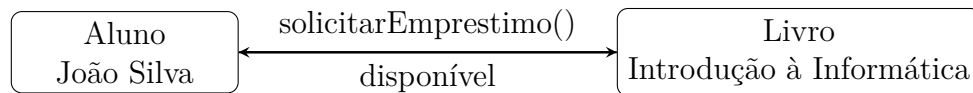
Classe	Objeto
Aluno	João Silva
Livro	Introdução à Informática

4. Exemplo de troca de mensagens entre objetos

Imagine que João deseja emprestar um livro.

O objeto **Aluno** pode solicitar o empréstimo ao sistema. Em seguida, o sistema verifica se o objeto **Livro** está disponível.

Uma representação simplificada seria:



Nesse exemplo, o aluno solicita o empréstimo e o livro informa sua disponibilidade.

5. Principal função do sistema

A principal função do sistema é controlar o empréstimo e a devolução dos livros da biblioteca escolar.

Além disso, o sistema poderá auxiliar no registro dos alunos, na consulta dos livros disponíveis e na identificação dos livros que estão emprestados.

Capítulo 2

Processo de Desenvolvimento de Software

Desenvolver um sistema de software envolve mais do que escrever código. Antes da programação, é necessário compreender o problema, identificar as necessidades dos usuários e planejar como o sistema será construído. Depois da implementação, também é necessário testar o software e disponibilizá-lo aos usuários. Esse conjunto de atividades faz parte do **processo de desenvolvimento de software**.

Neste capítulo, estudaremos seis atividades importantes desse processo:

- Levantamento de requisitos;
- Análise;
- Projeto;
- Implementação;
- Testes;
- Implantação.

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- Compreender o que é um processo de desenvolvimento de software;
- Identificar as principais atividades realizadas durante o desenvolvimento de um sistema;
- Compreender a finalidade do levantamento de requisitos;
- Diferenciar análise e projeto;
- Compreender o papel da implementação e dos testes;
- Identificar as principais atividades relacionadas à implantação de um sistema.

2.1 O que é um Processo de Desenvolvimento de Software?

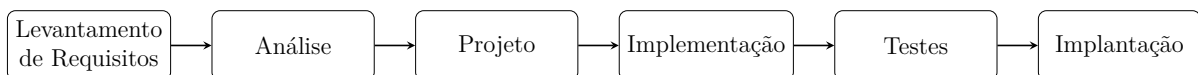
Conceito Central

Um **processo de desenvolvimento de software** organiza as atividades necessárias para compreender, construir, testar e disponibilizar um sistema de software.

Um sistema de software normalmente é construído para resolver um problema ou atender a uma necessidade.

Por exemplo, uma escola pode precisar controlar os empréstimos realizados em sua biblioteca. Embora seja possível começar imediatamente a escrever um programa, essa abordagem pode produzir um sistema que não atenda corretamente às necessidades da escola. Por isso, o desenvolvimento é organizado em atividades.

De maneira simplificada, podemos representar essas atividades da seguinte forma:



Essa representação ajuda a compreender a finalidade de cada atividade. Entretanto, na prática, o desenvolvimento nem sempre ocorre de forma totalmente linear. Algumas atividades podem ser repetidas ou realizadas novamente quando novas necessidades ou problemas são identificados.

2.2 Levantamento de Requisitos

O primeiro passo é compreender o problema que precisa ser resolvido. O **levantamento de requisitos** busca identificar as necessidades dos usuários e aquilo que o sistema deverá oferecer.

Durante essa atividade, a equipe de desenvolvimento procura conversar com usuários e especialistas, observar como as atividades são realizadas e conhecer as informações utilizadas no ambiente.

Conceito Central

Um **requisito** representa uma necessidade, condição ou capacidade que deve ser atendida pelo sistema.

Considere novamente o sistema da biblioteca escolar. Durante o levantamento de requisitos, algumas perguntas podem ser realizadas:

- Quem utilizará o sistema?
- Como os livros são cadastrados?
- Como os empréstimos são realizados atualmente?

- Qual é o prazo para devolução?
- Um aluno pode emprestar quantos livros?
- Como identificar livros disponíveis?

A partir das respostas, podem ser identificados requisitos como:

- o sistema deve permitir cadastrar livros;
- o sistema deve permitir cadastrar alunos;
- o sistema deve registrar empréstimos;
- o sistema deve registrar devoluções;
- o sistema deve permitir consultar os livros disponíveis.

Observe que, nesta atividade, o interesse principal está em descobrir **o que os usuários precisam**.

Pergunta-chave

Levantamento de requisitos: O que o usuário precisa?

2.2.1 Requisitos Funcionais e Não Funcionais

Os requisitos podem representar diferentes aspectos do sistema. Os **requisitos funcionais** descrevem funções ou serviços que o sistema deverá oferecer. Exemplos:

- cadastrar um livro;
- registrar um empréstimo;
- consultar um aluno;
- registrar uma devolução.

Os **requisitos não funcionais** estão relacionados a características ou restrições do sistema, como segurança, desempenho, facilidade de uso e plataformas utilizadas. Por exemplo:

- o sistema deve exigir autenticação do bibliotecário;
- o sistema deve apresentar uma interface de fácil utilização;
- as consultas devem apresentar os resultados rapidamente.

2.3 Análise

Depois de levantar os requisitos, é necessário estudá-los com mais detalhes. Essa atividade é chamada de **análise**. Durante a análise, a equipe procura compreender como os elementos do problema estão relacionados e constrói modelos que representam o sistema que deverá ser desenvolvido. No sistema da biblioteca, a análise pode identificar elementos importantes, como:

- Aluno;
- Livro;
- Bibliotecário;
- Empréstimo;
- Devolução.

Também podem ser identificadas regras, como:

- um livro não pode ser emprestado se estiver indisponível;
- um empréstimo deve estar associado a um aluno;
- cada empréstimo deve possuir uma data de realização;
- uma devolução deve atualizar a disponibilidade do livro.

Nesta etapa, o objetivo ainda não é escolher uma linguagem de programação, um banco de dados ou um servidor específico. O interesse principal é compreender e representar corretamente **o que o sistema deverá fazer**.

Pergunta-chave

Análise: O que o sistema deve fazer?

2.4 Projeto

Depois de compreender o que o sistema deverá fazer, é necessário decidir **como ele será construído**. Essa atividade é chamada de **projeto de software**. No projeto, são consideradas decisões relacionadas à tecnologia e à organização da solução. Entre essas decisões podem estar:

- organização dos componentes do sistema;
- estrutura do banco de dados;
- organização da interface gráfica;
- algoritmos necessários;

- comunicação entre diferentes partes do sistema;
- distribuição do software nos equipamentos.

No sistema da biblioteca escolar, por exemplo, podem ser tomadas decisões como:

- utilizar uma aplicação Web;
- armazenar alunos e livros em um banco de dados;
- criar uma tela para cadastro de livros;
- criar uma tela para empréstimos;
- criar uma tela para consultas;
- definir como as diferentes partes do sistema trocarão informações.

Pergunta-chave

Projeto: Como o sistema será construído?

2.5 Análise ou Projeto?

É comum que estudantes confundam essas duas atividades. Uma forma simples de diferenciá-las é observar que a análise está mais relacionada ao problema, enquanto o projeto está relacionado à solução computacional.

	Análise	Projeto
Pergunta principal	O que o sistema deve fazer?	Como o sistema será construído?
Foco	Problema e necessidades	Solução computacional
Exemplos	Identificar livros, alunos, empréstimos e regras	Definir interface, banco de dados e organização do sistema

Considere o seguinte requisito:

O sistema deve permitir registrar o empréstimo de um livro.

Na **análise**, procura-se compreender:

- quem realiza o empréstimo;
- quais informações são necessárias;
- quais regras devem ser verificadas.

No **projeto**, são definidas questões como:

- qual tela será utilizada;
- como os dados serão armazenados;
- quais componentes executarão a operação.

2.6 Implementação

Depois que a solução foi analisada e projetada, inicia-se a construção do software. Essa etapa é chamada de **implementação**.

Na implementação, as definições produzidas durante o projeto são transformadas em código utilizando uma ou mais linguagens de programação.

Conceito Central

Implementar significa transformar o projeto do sistema em um software executável.

No sistema da biblioteca, durante a implementação podem ser desenvolvidos:

- o cadastro de alunos;
- o cadastro de livros;
- o registro de empréstimos;
- o registro de devoluções;
- as consultas ao acervo;
- a interface utilizada pelos usuários.

Bibliotecas de programação, componentes e outros recursos de software também podem ser reutilizados durante essa atividade.

2.7 Testes

Um programa estar implementado não significa que esteja pronto para ser utilizado. É necessário verificar se o software funciona corretamente e se atende aos requisitos definidos. Essa verificação é realizada por meio de **testes**. Considere o seguinte requisito:

O sistema deve permitir o empréstimo apenas de livros disponíveis.

Durante os testes, podemos verificar situações como:

- emprestar um livro disponível;
- tentar emprestar um livro que já está emprestado;
- devolver um livro;
- verificar se o livro voltou a ficar disponível.

Se o comportamento encontrado for diferente do comportamento esperado, o problema deve ser identificado e corrigido.

2.7.1 Tipos de Testes

Os testes podem ser realizados em diferentes níveis.

Tipo	Objetivo
Teste de unidade	Verificar pequenas partes do software, como métodos ou classes.
Teste de integração	Verificar se diferentes partes do sistema funcionam corretamente quando trabalham juntas.
Teste de sistema	Verificar o funcionamento do sistema completo em relação aos requisitos definidos.
Teste de aceitação	Verificar, juntamente com os usuários, se o sistema atende às necessidades esperadas.

Não é necessário decorar os diferentes tipos neste momento. O mais importante é compreender que os testes permitem encontrar problemas antes que eles prejudiquem os usuários.

Pergunta-chave

Testes: O sistema funciona conforme o esperado?

2.8 Implantação

Após a construção e os testes, o sistema precisa ser disponibilizado para uso. Essa atividade é chamada de **implantação**. Durante a implantação, o software pode ser preparado, instalado e configurado no ambiente em que será utilizado. Também podem ser realizadas atividades como:

- instalação do sistema;
- configuração dos equipamentos;
- importação de dados existentes;
- criação ou preparação de manuais;
- treinamento dos usuários;
- migração de dados de um sistema anterior.

No caso da biblioteca, por exemplo, pode ser necessário importar para o novo sistema a lista de alunos e os livros que já estavam cadastrados em planilhas ou registros anteriores. Depois disso, os bibliotecários podem receber treinamento para utilizar as novas funcionalidades.

Pergunta-chave

Implantação: Como disponibilizar o sistema para seus usuários?

2.9 Visão Geral do Processo

Podemos resumir as atividades estudadas por meio das perguntas que cada uma procura responder.

Atividade	Pergunta principal
Levantamento de requisitos	O que os usuários precisam?
Análise	O que o sistema deve fazer?
Projeto	Como o sistema será construído?
Implementação	Como transformar o projeto em código executável?
Testes	O sistema funciona conforme foi especificado?
Implantação	Como disponibilizar o sistema para os usuários?

2.10 Exemplo Integrador: Biblioteca Escolar

Vamos acompanhar como o sistema da biblioteca pode passar pelas diferentes atividades do processo de desenvolvimento.

Atividade	Exemplo no Sistema de Biblioteca
Levantamento de requisitos	Conversar com bibliotecários e alunos para identificar necessidades, como cadastrar livros e controlar empréstimos.
Análise	Identificar elementos como Aluno, Livro e Empréstimo, além das regras de funcionamento da biblioteca.
Projeto	Definir a interface, a organização do banco de dados e a estrutura do software.
Implementação	Programar os cadastros, consultas, empréstimos e devoluções.
Testes	Verificar se os empréstimos, devoluções e consultas funcionam corretamente.
Implantação	Instalar o sistema, importar os dados dos livros e treinar os bibliotecários.

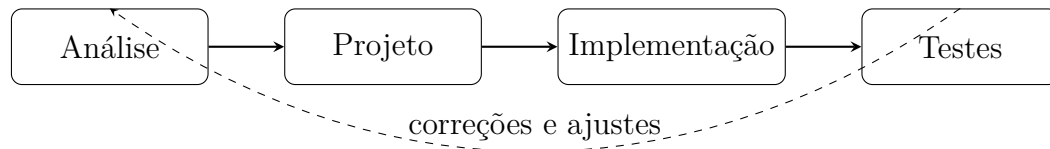
Assim, uma necessidade inicial é transformada gradualmente em um sistema que pode ser utilizado pelos usuários:

Necessidade → Requisitos → Análise → Projeto → Software

2.11 Importante: o Processo Nem Sempre é Linear

A sequência apresentada neste capítulo é útil para compreender a finalidade das atividades de desenvolvimento. Entretanto, projetos reais podem exigir o retorno a atividades anteriores.

Durante os testes, por exemplo, pode ser identificado que um requisito não foi compreendido corretamente. Nesse caso, pode ser necessário revisar a análise, alterar o projeto e modificar a implementação.



Nos próximos capítulos serão estudadas diferentes formas de organizar essas atividades ao longo do desenvolvimento de um software.

2.12 Resumo do Capítulo

Neste capítulo, vimos que o desenvolvimento de um software pode ser organizado em diferentes atividades.

O **levantamento de requisitos** procura identificar as necessidades dos usuários.

Na **análise**, essas necessidades são estudadas e representadas de maneira mais detalhada.

No **projeto**, são definidas as soluções computacionais necessárias para construir o sistema.

Na **implementação**, o projeto é transformado em código executável.

Os **testes** verificam se o software funciona corretamente e atende aos requisitos.

Finalmente, na **implantação**, o sistema é preparado e disponibilizado para seus usuários.

Essas atividades estão relacionadas e podem ser realizadas mais de uma vez durante o desenvolvimento de um sistema.

2.13 Fixando os Conceitos

1. O que é um processo de desenvolvimento de software?
2. Qual é o principal objetivo do levantamento de requisitos?
3. Explique a diferença entre análise e projeto.
4. Em qual atividade são escolhidas soluções relacionadas à interface, banco de dados e organização do software?
5. O que acontece durante a implementação?
6. Por que um sistema precisa ser testado?
7. Qual é a diferença entre teste de unidade e teste de sistema?
8. O que significa implantar um sistema?
9. Cite três atividades que podem ser realizadas durante a implantação.
10. Por que as atividades do desenvolvimento nem sempre ocorrem de maneira totalmente linear?

2.14 Atividade Prática

Considere que a escola deseja desenvolver um **Sistema para a Cantina Escolar**. O sistema deverá permitir controlar produtos e registrar as vendas realizadas. Com base no conteúdo deste capítulo, responda:

1. Cite cinco perguntas que poderiam ser realizadas durante o levantamento de requisitos.
2. Identifique quatro possíveis requisitos do sistema.
3. Explique o que deveria ser estudado durante a análise.
4. Cite três decisões que poderiam ser tomadas durante o projeto.
5. Cite três funcionalidades que precisariam ser implementadas.
6. Descreva três situações que deveriam ser testadas.
7. Cite três atividades que poderiam ser realizadas durante a implantação do sistema.

Capítulo 3

O Componente Humano no Desenvolvimento de Software

No capítulo anterior, vimos que o desenvolvimento de um software envolve atividades como levantamento de requisitos, análise, projeto, implementação, testes e implantação. Essas atividades não são realizadas apenas por computadores ou ferramentas. Elas dependem principalmente das **pessoas** envolvidas no projeto.

O desenvolvimento de software é uma atividade colaborativa. Dependendo do tamanho e da complexidade do sistema, profissionais com diferentes conhecimentos e responsabilidades podem participar de sua construção. Entre esses profissionais estão:

- gerente de projetos;
- analista de sistemas;
- projetista de sistemas;
- arquiteto de software;
- programador;
- especialista do domínio;
- avaliador de qualidade.

Neste capítulo, estudaremos a função de cada um desses participantes e como eles colaboram durante o desenvolvimento de um sistema.

Objetivos de Aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- compreender por que o desenvolvimento de software é um trabalho realizado em equipe;
- identificar os principais profissionais envolvidos no desenvolvimento;
- reconhecer as responsabilidades do gerente de projetos;
- compreender o papel do analista na comunicação com os usuários;

- diferenciar as funções do analista, projetista, arquiteto e programador;
- compreender a importância do especialista do domínio;
- reconhecer a função da avaliação de qualidade;
- relacionar os diferentes profissionais às atividades do processo de desenvolvimento.

3.1 Software é Construído por Pessoas

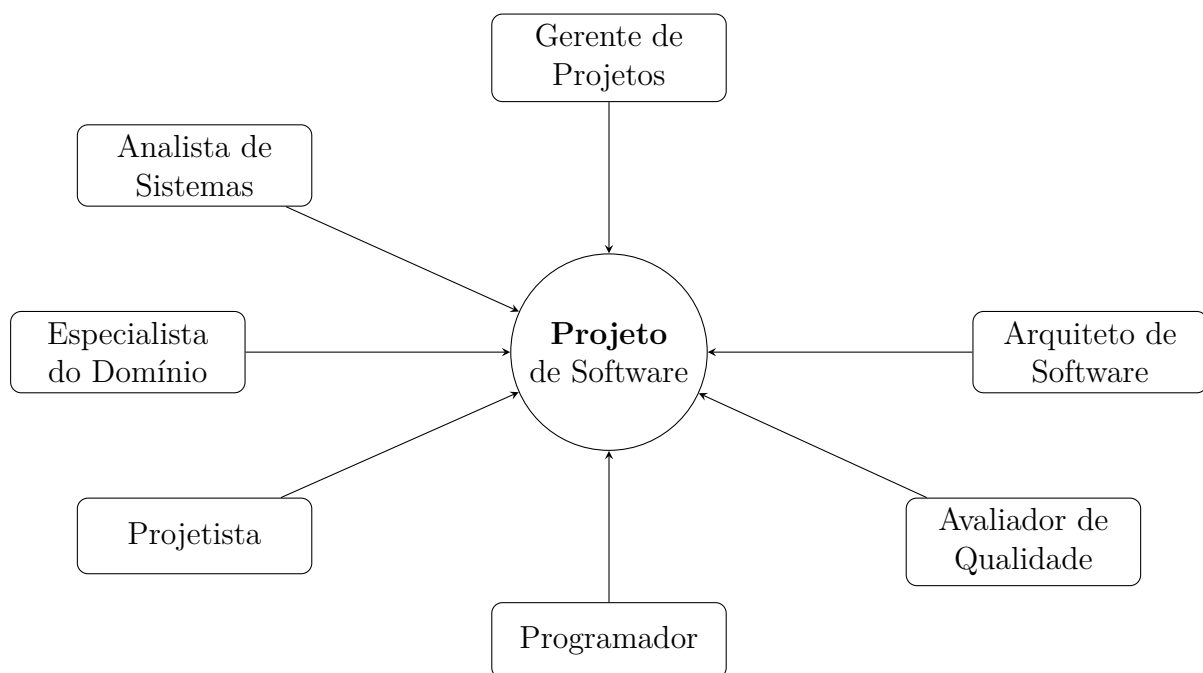
Um sistema pode envolver diferentes tecnologias, como linguagens de programação, bancos de dados, redes, servidores e dispositivos móveis. Entretanto, são as pessoas que identificam necessidades, tomam decisões, criam modelos, escrevem programas, realizam testes e avaliam se o sistema atende aos seus usuários. Por isso, o desenvolvimento de software normalmente envolve uma **equipe de desenvolvimento**.

Conceito Central

Uma **equipe de desenvolvimento de software** é formada por pessoas que colaboram para planejar, analisar, projetar, construir, testar e disponibilizar um sistema.

Em projetos pequenos, uma mesma pessoa pode desempenhar várias funções. Por exemplo, um profissional pode participar do levantamento dos requisitos, produzir modelos e também programar parte do sistema.

Em projetos maiores, essas responsabilidades podem ser distribuídas entre vários profissionais especializados.



A figura mostra que diferentes profissionais podem contribuir para um mesmo projeto. O resultado depende da colaboração entre todos eles.

3.2 Gerente de Projetos

O **gerente de projetos** é o profissional responsável por coordenar as atividades necessárias ao desenvolvimento do sistema. Ele procura garantir que o trabalho seja organizado e que os recursos disponíveis sejam utilizados adequadamente. Entre suas responsabilidades estão:

- organizar as atividades do projeto;
- definir e acompanhar o cronograma;
- estimar o tempo necessário para o desenvolvimento;
- acompanhar custos e recursos;
- organizar a equipe;
- acompanhar o andamento das atividades;
- auxiliar na definição do processo de desenvolvimento;
- verificar dificuldades e riscos que possam afetar o projeto.

Ideia Principal

O gerente de projetos procura responder:

Como organizar pessoas, recursos, atividades e prazos para que o projeto seja realizado?

3.2.1 Exemplo

Considere o desenvolvimento do Sistema de Biblioteca Escolar. O gerente pode organizar um planejamento como:

- Semana 1: levantamento dos requisitos;
- Semana 2: análise do sistema;
- Semana 3: elaboração do projeto;
- Semanas 4 e 5: implementação;
- Semana 6: testes;
- Semana 7: implantação.

Durante o desenvolvimento, ele acompanha se as atividades estão ocorrendo conforme o planejado.

3.3 Analista de Sistemas

O **analista de sistemas** procura compreender o problema que deverá ser resolvido pelo software. Para isso, precisa conversar com usuários e especialistas, conhecer o funcionamento da organização e identificar as necessidades que deverão ser atendidas pelo sistema. O analista funciona como uma ligação entre dois grupos:



O analista precisa compreender a linguagem utilizada pelos usuários e transformar suas necessidades em informações que possam ser utilizadas pela equipe técnica. Entre suas atividades estão:

- conversar com usuários;
- compreender o problema;
- identificar necessidades;
- definir e analisar requisitos;
- compreender regras do negócio;
- construir modelos do sistema;
- comunicar as necessidades aos demais profissionais.

Além do conhecimento técnico, a comunicação é uma característica importante para esse profissional.

Ideia Principal

O analista procura responder:

Qual é o problema e o que o sistema precisa fazer para resolvê-lo?

3.3.1 Exemplo

No Sistema de Biblioteca Escolar, o analista pode conversar com o bibliotecário e perguntar:

- Como um empréstimo é realizado?
- Quantos livros um aluno pode emprestar?
- Qual é o prazo de devolução?
- Como saber se um livro está disponível?
- Quem pode cadastrar novos livros?

Com essas informações, o analista poderá identificar os requisitos e construir modelos para representar o sistema.

3.4 Projetista de Sistemas

Depois que o problema foi analisado, é necessário definir uma solução computacional. Essa é uma das principais responsabilidades do **projetista de sistemas**.

O projetista trabalha a partir dos resultados produzidos durante a análise e acrescenta decisões relacionadas à tecnologia que será utilizada. Entre suas atividades podem estar:

- avaliar alternativas de solução;
- detalhar como determinadas partes do sistema serão construídas;
- definir aspectos da interface;
- definir aspectos do banco de dados;
- definir aspectos de comunicação e redes;
- detalhar modelos que serão utilizados na implementação.

Podem existir diferentes especializações, como:

- projetista de interface;
- projetista de banco de dados;
- projetista de redes.

Ideia Principal

O projetista procura responder:

Como transformar os resultados da análise em uma solução computacional detalhada?

3.4.1 Exemplo

No Sistema de Biblioteca Escolar, durante a análise foi identificado que:

O sistema deve permitir registrar empréstimos.

O projetista poderá definir:

- como será a tela de empréstimo;
- quais dados deverão ser informados;
- como o empréstimo será armazenado;
- quais tabelas do banco de dados serão utilizadas;
- como o sistema verificará se o livro está disponível.

3.5 Arquiteto de Software

Em sistemas maiores e mais complexos, pode existir um profissional responsável por definir a organização geral da solução: o **arquiteto de software**. Enquanto o projetista pode concentrar-se em partes específicas do sistema, o arquiteto observa a solução de maneira mais ampla. Ele pode decidir:

- quais partes ou subsistemas formarão o sistema;
- como essas partes estarão organizadas;
- como os componentes se comunicarão;
- quais decisões técnicas afetam o sistema como um todo;
- como aspectos como desempenho podem ser considerados.



A figura apresenta um exemplo muito simples de organização do sistema em partes. Em sistemas reais, a arquitetura pode ser consideravelmente mais complexa.

Ideia Principal

O arquiteto procura responder:

Como organizar a estrutura geral do sistema e a comunicação entre suas principais partes?

3.6 Programador

O **programador** é o profissional responsável pela implementação do sistema. Ele utiliza linguagens de programação e outras tecnologias para transformar os modelos e especificações do projeto em software executável. Um programador pode trabalhar com:

- linguagens de programação;
- bancos de dados;
- bibliotecas de software;
- frameworks;
- interfaces;
- APIs;
- ferramentas de desenvolvimento.

Também precisa compreender os modelos e as especificações produzidas durante as etapas anteriores.

Ideia Principal

O programador procura responder:

Como transformar o projeto em código que possa ser executado pelo computador?

3.6.1 Exemplo

No Sistema de Biblioteca Escolar, o programador pode implementar:

- a tela de cadastro de livros;
- a consulta ao acervo;
- o registro de alunos;
- o registro de empréstimos;
- o registro de devoluções;
- a comunicação com o banco de dados.

3.7 Especialista do Domínio

Nem todas as pessoas importantes para a construção de um sistema são profissionais de informática. O **especialista do domínio** é a pessoa que possui conhecimento sobre a área na qual o sistema será utilizado.

O termo **domínio** representa a área de conhecimento ou de atividade relacionada ao problema que está sendo estudado. Alguns exemplos são:

Sistema	Possível Especialista do Domínio
Sistema de biblioteca	Bibliotecário
Sistema escolar	Professor, secretário ou coordenador
Sistema hospitalar	Médico, enfermeiro ou profissional administrativo
Sistema de oficina mecânica	Mecânico ou responsável pela oficina
Sistema de vendas	Vendedor ou gerente comercial

O especialista ajuda a equipe a compreender:

- como o trabalho é realizado atualmente;
- quais informações são importantes;
- quais regras devem ser respeitadas;
- quais problemas precisam ser resolvidos;
- quais necessidades os usuários possuem.

Ideia Principal

O especialista do domínio procura responder:
Como essa atividade funciona no mundo real?

3.7.1 Analista e Especialista do Domínio

Esses dois participantes trabalham de maneira próxima, mas suas funções são diferentes.

Profissional	Principal contribuição
Especialista do domínio	Conhece profundamente o negócio ou área onde o sistema será utilizado.
Analista de sistemas	Obtém esse conhecimento e o transforma em requisitos e modelos para orientar o desenvolvimento.

No exemplo da biblioteca, o bibliotecário sabe como os empréstimos funcionam. O analista utiliza esse conhecimento para definir como essas necessidades serão representadas no sistema.

3.8 Avaliador de Qualidade

Durante o desenvolvimento, não basta apenas construir o software. É necessário verificar se o processo e o produto atendem aos padrões de qualidade estabelecidos. Essa é uma das responsabilidades do **avaliador de qualidade**.

Características como confiabilidade e desempenho podem fazer parte da qualidade esperada de um software. O avaliador pode verificar se:

- os procedimentos definidos estão sendo seguidos;
- a documentação está adequada;
- os requisitos estão sendo considerados;
- os testes estão sendo realizados;
- o produto atende aos critérios de qualidade definidos.

Ideia Principal

O avaliador de qualidade procura responder:
O processo e o software produzido atendem aos padrões de qualidade estabelecidos?

3.9 Os Profissionais Trabalham em Conjunto

Embora cada profissional possua responsabilidades específicas, o desenvolvimento de software depende da comunicação entre eles. Podemos observar essa colaboração em um exemplo simplificado:



O gerente de projetos acompanha e organiza as atividades realizadas ao longo desse processo. Essa figura é uma simplificação didática. Na prática, a comunicação pode ocorrer em várias direções e os profissionais trabalham de forma colaborativa.

3.10 Quem Faz o Quê?

A tabela a seguir resume as funções estudadas.

Profissional	Principal responsabilidade	Pergunta-chave
Gerente de projetos	Organizar e acompanhar pessoas, recursos, atividades e prazos.	Como organizar o projeto?
Analista	Compreender necessidades e transformá-las em requisitos e modelos.	O que o sistema precisa fazer?
Projetista	Detalhar a solução computacional e acrescentar decisões tecnológicas.	Como a solução será construída?
Arquiteto de software	Definir a organização geral do sistema e de seus componentes.	Como organizar o sistema como um todo?
Programador	Transformar o projeto em código executável.	Como implementar a solução?
Especialista do domínio	Fornecer conhecimento sobre a área ou negócio.	Como o negócio funciona?
Avaliador de qualidade	Verificar a adequação do processo e do produto aos padrões de qualidade.	O software possui a qualidade esperada?

3.11 Exemplo Integrador: Biblioteca Escolar

Vamos considerar novamente o desenvolvimento do Sistema de Biblioteca Escolar. Cada participante pode contribuir de maneira diferente.

Participante	Exemplo de Atividade
Gerente de projetos	Organiza a equipe, o cronograma e acompanha o desenvolvimento.
Analista	Conversa com os bibliotecários e identifica os requisitos do sistema.
Projetista	Define como serão as telas, os dados e outros detalhes da solução.
Arquiteto	Define como as principais partes do sistema serão organizadas e integradas.
Programador	Implementa cadastros, consultas, empréstimos e devoluções.
Especialista do domínio	O bibliotecário explica como funciona o controle dos livros e quais regras devem ser seguidas.
Avaliador de qualidade	Verifica se o processo e o sistema atendem aos critérios de qualidade definidos.

3.12 Importante: Papéis não são Pessoas

Os nomes apresentados neste capítulo representam **papéis ou responsabilidades** dentro de um projeto. Isso significa que não é necessário existir sempre uma pessoa diferente para cada papel. Em uma pequena equipe, uma pessoa pode atuar como analista, projetista e programador. Em um projeto maior, vários profissionais podem trabalhar como analistas ou programadores.

Importante

Papel indica uma responsabilidade no projeto. Uma pessoa pode desempenhar vários papéis e um mesmo papel pode ser desempenhado por várias pessoas.

3.13 Resumo do Capítulo

O desenvolvimento de software é uma atividade realizada em equipe.

O **gerente de projetos** coordena e acompanha o desenvolvimento.

O **analista** procura compreender as necessidades dos usuários e transformá-las em requisitos e modelos.

O **projetista** acrescenta detalhes tecnológicos e especifica como a solução será construída.

O **arquiteto de software** define a organização geral do sistema e a comunicação entre suas principais partes.

O **programador** transforma as especificações em código executável.

O **especialista do domínio** fornece o conhecimento sobre a área na qual o software será utilizado.

O **avaliador de qualidade** verifica a adequação do processo e do produto aos padrões de qualidade estabelecidos.

Embora cada papel possua responsabilidades diferentes, todos precisam colaborar para que o sistema desenvolvido atenda às necessidades dos usuários.

3.14 Fixando os Conceitos

1. Por que o desenvolvimento de software é considerado uma atividade realizada em equipe?
2. Qual é a principal função do gerente de projetos?
3. Qual é o papel do analista na comunicação entre usuários e desenvolvedores?
4. Explique a diferença entre analista e projetista.
5. Qual é a função de um arquiteto de software?
6. Qual é a principal responsabilidade do programador?
7. O que significa especialista do domínio?
8. Em um sistema hospitalar, quem poderia atuar como especialista do domínio?
9. Qual é o objetivo do avaliador de qualidade?
10. Uma mesma pessoa pode desempenhar mais de uma função no projeto? Explique.

3.15 Atividade Prática

Considere que uma escola deseja desenvolver um **Sistema para a Cantina Escolar**. O sistema deverá permitir cadastrar produtos, controlar o estoque e registrar vendas. Imagine que você precisa montar uma pequena equipe para desenvolver esse sistema. Responda:

1. Quem poderia atuar como especialista do domínio? Explique.
2. Cite duas atividades que seriam responsabilidade do gerente de projetos.
3. Cite três perguntas que o analista poderia fazer ao responsável pela cantina.
4. Cite duas decisões que poderiam ser tomadas pelo projetista.
5. Qual seria uma possível responsabilidade do arquiteto de software?
6. Cite três funcionalidades que o programador poderia implementar.
7. Cite duas verificações que poderiam ser realizadas pelo avaliador de qualidade.
8. Se a equipe tivesse apenas três profissionais de informática, quais papéis poderiam ser acumulados por uma mesma pessoa?

Capítulo 4

Modelos de Ciclo de Vida de Software

Nos capítulos anteriores, estudamos as principais atividades realizadas durante o desenvolvimento de software e os profissionais que participam desse processo. Entretanto, ainda precisamos responder a uma questão:

Como organizar as atividades de levantamento de requisitos, análise, projeto, implementação, testes e implantação durante o desenvolvimento de um software?

Existem diferentes maneiras de organizar essas atividades. Essas formas de organização são estudadas por meio dos **modelos de processo** e dos **modelos de ciclo de vida de software**. Neste capítulo, estudaremos diferentes abordagens, desde modelos mais sequenciais até métodos iterativos e ágeis.

Objetivos de Aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- compreender o conceito de modelo de ciclo de vida;
- reconhecer as características dos modelos prescritivos;
- compreender o funcionamento do modelo Cascata;
- compreender o desenvolvimento Incremental;
- identificar as características do RAD;
- compreender a finalidade da Prototipagem;
- reconhecer a importância da análise de riscos no modelo Espiral;
- identificar as fases do Processo Unificado;
- compreender os princípios básicos da Extreme Programming (XP);
- reconhecer os principais elementos do Scrum;
- comparar diferentes formas de organizar um projeto de software.

4.1 O que é um Modelo de Ciclo de Vida?

O desenvolvimento de um software envolve diversas atividades, como levantamento de requisitos, análise, projeto, implementação, testes e implantação. Um **modelo de ciclo de vida** determina como essas atividades serão organizadas ao longo do projeto.

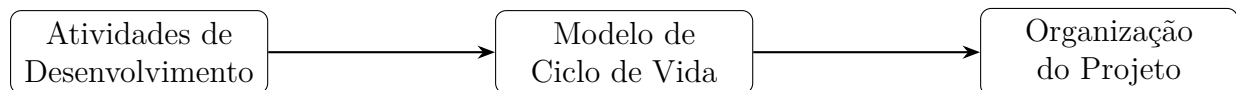
Conceito Central

Um **modelo de ciclo de vida de software** representa uma forma de organizar e relacionar as atividades realizadas durante o desenvolvimento de um sistema.

Dependendo do modelo adotado, as atividades podem ocorrer:

- uma após a outra;
- em pequenos ciclos;
- em diferentes versões do sistema;
- com construção de protótipos;
- com análise contínua de riscos;
- com entregas frequentes ao usuário.

Portanto, diferentes modelos podem organizar as mesmas atividades de maneiras diferentes.



4.2 Modelos Prescritivos de Processo

Os **modelos prescritivos de processo** procuram fornecer uma estrutura para organizar o desenvolvimento de software. Eles descrevem atividades, etapas e formas de relacionamento entre essas atividades. Isso não significa necessariamente que todo modelo prescritivo seja rígido ou totalmente sequencial. Existem modelos:

- sequenciais;
- incrementais;
- evolucionários;
- orientados a riscos.

Cada abordagem procura solucionar determinados problemas encontrados durante o desenvolvimento.

Importante

Não existe um único modelo adequado para todos os projetos. A escolha depende de fatores como tamanho do sistema, estabilidade dos requisitos, riscos, tecnologias utilizadas, prazo e participação dos usuários.

4.3 Modelo Cascata

O **modelo Cascata**, também chamado de modelo clássico ou linear, organiza as atividades de forma predominantemente sequencial. Nesse modelo, uma etapa é normalmente concluída antes do início da etapa seguinte. Uma representação simplificada é apresentada na Figura 4.1.

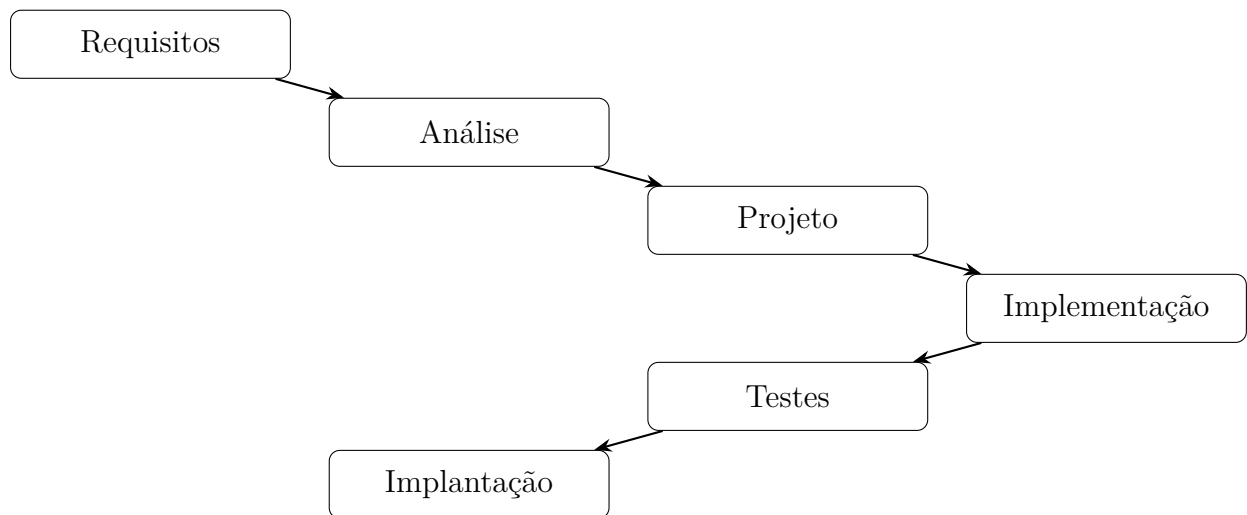


Figura 4.1: Representação didática do modelo Cascata.

A ideia principal é semelhante a uma cascata: o desenvolvimento progride de uma etapa para a seguinte.

4.3.1 Exemplo

Considere o Sistema de Biblioteca Escolar. No modelo Cascata, a equipe poderia realizar:

1. levantar todos os requisitos;
2. analisar completamente o sistema;
3. elaborar o projeto;
4. implementar o software;
5. realizar os testes;
6. implantar o sistema.

4.3.2 Vantagens

Entre as características favoráveis estão:

- organização simples;
- etapas bem definidas;
- facilidade para acompanhar o avanço das fases;
- documentação produzida ao longo do processo.

4.3.3 Limitações

O modelo pode apresentar dificuldades quando:

- os requisitos mudam com frequência;
- o usuário precisa visualizar o sistema rapidamente;
- erros dos requisitos são percebidos somente nas últimas etapas;
- o projeto precisa produzir versões funcionais antecipadamente.

Ideia Principal

No modelo Cascata, o projeto segue predominantemente uma **sequência de etapas**.

4.4 Modelo Incremental

No **modelo Incremental**, o software é desenvolvido e entregue em partes chamadas **incrementos**. Em vez de esperar que todo o sistema esteja pronto, algumas funcionalidades são desenvolvidas primeiro. Outras funcionalidades são adicionadas nas versões seguintes.

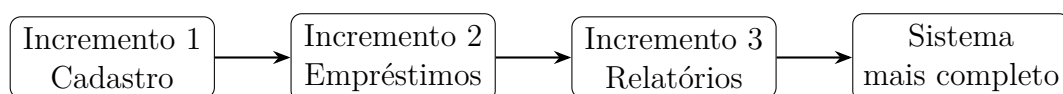


Figura 4.2: Desenvolvimento do sistema por incrementos.

Por exemplo:

Incremento 1: cadastro de alunos e livros;

Incremento 2: controle de empréstimos e devoluções;

Incremento 3: consultas e relatórios.

Cada incremento aumenta as funcionalidades disponíveis no sistema.

Conceito Central

No desenvolvimento incremental, o software cresce por meio de **entregas sucessivas de funcionalidades**.

Entre suas vantagens estão:

- disponibilização antecipada de partes do sistema;
- possibilidade de obter feedback dos usuários;
- maior facilidade para priorizar funcionalidades;
- mudanças podem ser incorporadas em incrementos posteriores.

4.5 RAD – Rapid Application Development

RAD significa *Rapid Application Development*, ou **Desenvolvimento Rápido de Aplicações**. Essa abordagem procura reduzir o tempo necessário para disponibilizar uma aplicação, utilizando ciclos curtos, prototipagem, desenvolvimento iterativo e participação frequente dos usuários.

Na abordagem RAD, o usuário pode avaliar versões preliminares e fornecer *feedback* durante o desenvolvimento.

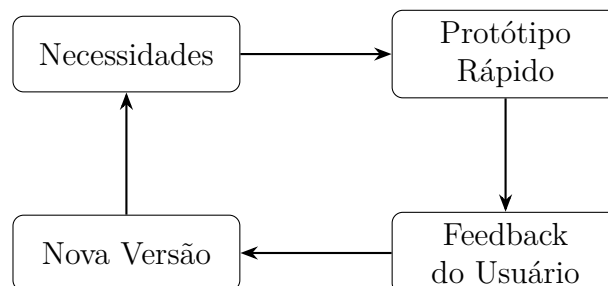


Figura 4.3: Representação simplificada do desenvolvimento rápido e iterativo.

Imagine que a equipe esteja desenvolvendo a tela de empréstimos da biblioteca. Em vez de produzir toda a aplicação antes de apresentá-la ao bibliotecário, a equipe pode desenvolver rapidamente uma versão da tela. O bibliotecário testa e faz sugestões. A equipe modifica a aplicação e apresenta uma nova versão.

Importante

RAD não significa simplesmente “programar rapidamente”. A abordagem depende principalmente de **iterações curtas, prototipagem e participação dos usuários**.

4.6 Prototipagem

Um **protótipo** é uma versão preliminar de um sistema ou de parte dele. A prototipagem é especialmente útil quando os usuários ainda não conseguem definir claramente todos os

requisitos. Ao visualizar e utilizar uma versão simplificada, o usuário pode compreender melhor o que necessita.

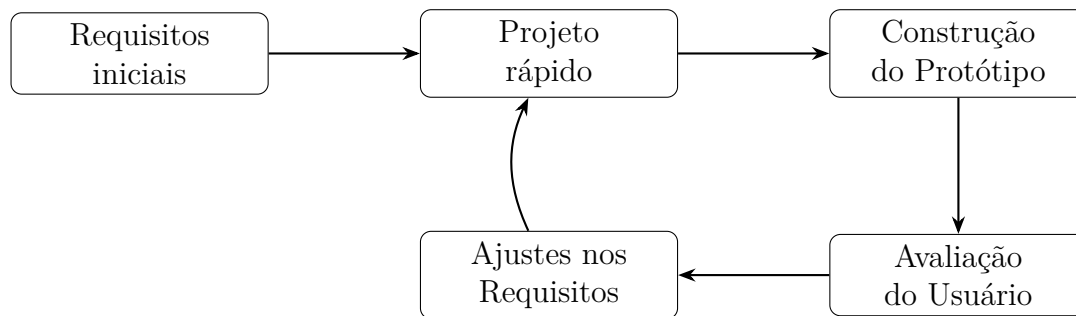


Figura 4.4: Ciclo simplificado da prototipagem.

O processo pode ocorrer várias vezes:

Construir → Avaliar → Corrigir → Construir novamente

4.6.1 Exemplo

Antes de implementar completamente o sistema da biblioteca, a equipe pode criar protótipos das telas de:

- cadastro de livros;
- empréstimo;
- devolução;
- consulta ao acervo.

Os bibliotecários avaliam essas telas e informam quais modificações são necessárias.

Ideia Principal

O protótipo ajuda a equipe e os usuários a **descobrir e compreender melhor os requisitos**.

4.7 Modelo Espiral

O **modelo Espiral** foi proposto por Barry Boehm e possui como uma de suas principais características a atenção aos **riscos do projeto**. O desenvolvimento ocorre em ciclos. Cada nova volta da espiral permite compreender melhor o problema, avaliar riscos e avançar no desenvolvimento do produto.

Em cada ciclo, a equipe pode:

1. definir objetivos;
2. identificar alternativas;

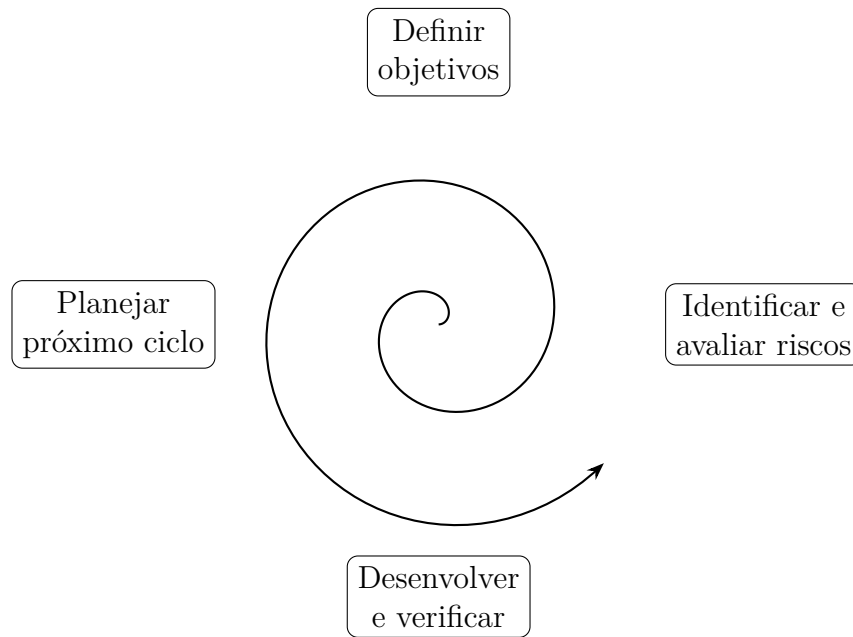


Figura 4.5: Representação didática simplificada do modelo Espiral.

3. analisar riscos;
4. desenvolver ou avaliar uma solução;
5. planejar o próximo ciclo.

4.7.1 O que é um risco?

Um risco é uma situação que pode prejudicar o projeto. Por exemplo:

- utilizar uma tecnologia que a equipe não conhece;
- existir uma grande possibilidade de mudança nos requisitos;
- utilizar um equipamento ainda não testado;
- existir dificuldade para atender aos requisitos de desempenho.

Conceito Central

A principal característica do modelo Espiral é utilizar os **riscos** como elemento importante para orientar o desenvolvimento.

4.8 Processo Unificado

O **Processo Unificado** utiliza desenvolvimento **iterativo e incremental**. Isso significa que o sistema é construído em várias iterações e cresce progressivamente. Uma implementação bastante conhecida dessa abordagem é o *Rational Unified Process* (RUP). Seu ciclo de vida pode ser organizado em quatro grandes fases:

1. Concepção;
2. Elaboração;
3. Construção;
4. Transição.



Figura 4.6: Fases e iterações do Processo Unificado.

4.8.1 Concepção

Nesta fase são definidos principalmente:

- objetivo do sistema;
- escopo inicial;
- principais requisitos;
- principais riscos;
- viabilidade do projeto.

4.8.2 Elaboração

Na elaboração, procura-se compreender melhor o problema e estabelecer uma base arquitetural para o sistema. Também são analisados os principais riscos.

4.8.3 Construção

Nesta fase ocorre grande parte da implementação. O software é desenvolvido gradualmente por meio das diferentes iterações.

4.8.4 Transição

A transição prepara o produto para utilização pelos usuários. Podem ocorrer:

- testes finais;
- correções;
- instalação;
- treinamento;
- disponibilização do sistema.

Ideia Principal

No Processo Unificado: uma fase pode possuir várias iterações e cada iteração produz avanços no sistema.

4.9 Desenvolvimento Ágil

Durante muitos projetos de software, os requisitos podem mudar enquanto o sistema ainda está sendo desenvolvido. Além disso, usuários frequentemente precisam utilizar partes do sistema antes de todo o produto estar concluído. Nesse contexto, abordagens ágeis valorizam:

- colaboração entre pessoas;
- participação do cliente;
- software funcionando;
- entregas frequentes;
- adaptação às mudanças.

Essas ideias foram consolidadas no **Manifesto para Desenvolvimento Ágil de Software**.

Importante

Agilidade não significa desenvolver software sem planejamento, documentação ou organização. Significa adaptar o processo, obter *feedback* frequentemente e produzir valor de maneira contínua.

4.10 Extreme Programming – XP

A **Extreme Programming (XP)** é uma abordagem ágil voltada especialmente às práticas de desenvolvimento de software. XP enfatiza valores como:

- comunicação;
- simplicidade;
- feedback;
- coragem;
- respeito.

Também utiliza práticas que procuram melhorar continuamente a qualidade do código e facilitar mudanças. Entre as práticas associadas à XP estão:

- pequenas entregas;

- programação em pares;
- testes frequentes;
- integração contínua;
- refatoração;
- projeto simples;
- participação próxima do cliente;
- propriedade coletiva do código.

4.10.1 Programação em Pares

Na programação em pares, dois desenvolvedores trabalham em conjunto. Enquanto um escreve o código, o outro acompanha, analisa e contribui para a solução. As funções podem ser trocadas durante o desenvolvimento.

4.10.2 Refatoração

Refatorar significa melhorar a estrutura interna do código sem alterar o comportamento esperado do software. O objetivo é manter o código:

- compreensível;
- organizado;
- simples;
- mais fácil de modificar.

4.10.3 Ciclo Simplificado de XP

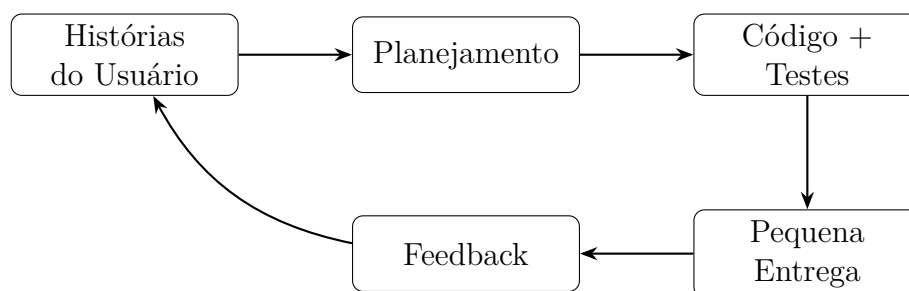


Figura 4.7: Representação didática de um ciclo de desenvolvimento em XP.

Ideia Principal

XP procura combinar entregas frequentes com **boas práticas técnicas de desenvolvimento**.

4.11 Scrum

O **Scrum** é um framework utilizado para organizar o trabalho em situações complexas. O trabalho é realizado em ciclos chamados **Sprints**. Uma Sprint possui duração fixa de **um mês ou menos**. Durante uma Sprint, a equipe trabalha para produzir um **Incremento** que represente valor para o produto.

4.11.1 Scrum Team

O Scrum define três responsabilidades principais:

- **Product Owner**;
- **Scrum Master**;
- **Developers**.

Esses participantes formam o **Scrum Team**.

4.11.2 Product Owner

O Product Owner procura maximizar o valor do produto e gerencia efetivamente o Product Backlog.

Entre suas responsabilidades está a organização e priorização do trabalho que precisa ser realizado.

4.11.3 Scrum Master

O Scrum Master ajuda a equipe a compreender e utilizar corretamente o Scrum. Também auxilia na remoção ou tratamento de obstáculos que dificultem o trabalho da equipe.

4.11.4 Developers

Os Developers são os profissionais responsáveis por realizar o trabalho necessário para criar um incremento utilizável do produto.

4.11.5 Artefatos

Entre os principais artefatos estão:

Product Backlog: lista organizada do trabalho necessário para melhorar o produto;

Sprint Backlog: trabalho selecionado para a Sprint;

Increment: resultado produzido durante o desenvolvimento.

4.11.6 Eventos

O Scrum possui eventos utilizados para organizar, inspecionar e adaptar o trabalho. São eles:

- Sprint;
- Sprint Planning;
- Daily Scrum;
- Sprint Review;
- Sprint Retrospective.

4.11.7 Fluxo Simplificado

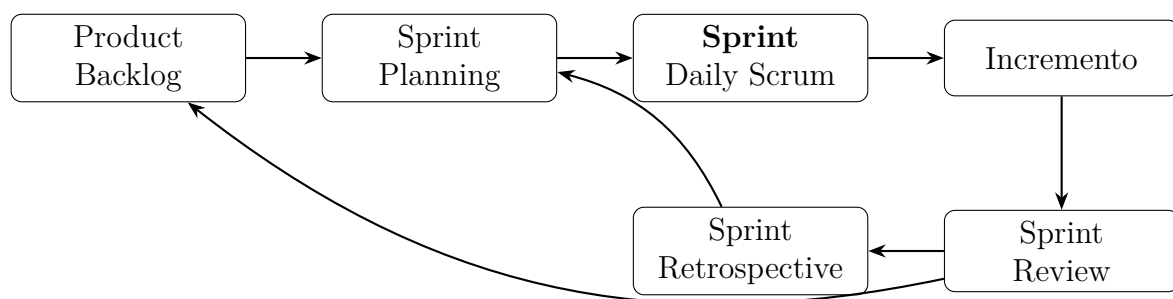


Figura 4.8: Representação simplificada do fluxo de trabalho no Scrum.

Ideia Principal

No Scrum, o trabalho ocorre em **Sprints**, nas quais a equipe procura produzir incrementos de valor, inspecionar os resultados e adaptar o trabalho seguinte.

4.12 XP e Scrum são a Mesma Coisa?

Não. Embora ambos estejam associados ao desenvolvimento ágil, possuem focos diferentes.

Aspecto	XP	Scrum
Foco principal	Práticas técnicas de desenvolvimento de software	Organização e gerenciamento do trabalho
Ciclos	Iterações e pequenas entregas	Sprints
Exemplos	Programação em pares, refatoração, testes e integração contínua	Sprint Planning, Daily Scrum, Review e Retrospective
Resultado	Software funcionando e tecnicamente sustentável	Incrementos produzidos durante as Sprints

XP e Scrum também podem ser utilizados de maneira complementar em determinados projetos.

4.13 Comparando os Modelos

A tabela a seguir resume as principais características estudadas.

Modelo	Característica Principal	Palavra-chave
Cascata	Etapas predominantemente sequenciais	Sequência
Incremental	O sistema cresce por meio de entregas sucessivas	Incrementos
RAD	Ciclos rápidos com intensa participação do usuário	Rapidez
Prototipagem	Construção de versões preliminares para compreender requisitos	Protótipo
Espiral	Desenvolvimento em ciclos orientados à análise de riscos	Riscos
Processo Unificado	Desenvolvimento iterativo e incremental organizado em fases	Iterações
XP	Desenvolvimento ágil com forte ênfase em práticas técnicas	Qualidade técnica
Scrum	Organização do trabalho em Sprints com inspeção e adaptação	Sprint

4.14 Exemplo Integrador: Biblioteca Escolar

Considere novamente o Sistema de Biblioteca Escolar. O mesmo sistema poderia ser desenvolvido utilizando diferentes abordagens.

Cascata: primeiro todos os requisitos seriam levantados e somente depois ocorreriam análise, projeto, implementação, testes e implantação.

Incremental: uma primeira versão poderia cadastrar livros, uma segunda controlar empréstimos e uma terceira gerar relatórios.

RAD: telas e funcionalidades poderiam ser desenvolvidas rapidamente e avaliadas pelos bibliotecários em ciclos curtos.

Prototipagem: protótipos das principais telas poderiam ser apresentados antes da implementação definitiva.

Espiral: a equipe poderia identificar riscos em cada ciclo, como segurança, desempenho ou uso de novas tecnologias.

Processo Unificado: o sistema seria desenvolvido em várias iterações distribuídas pelas fases de Concepção, Elaboração, Construção e Transição.

XP: a equipe poderia utilizar histórias de usuário, programação em pares, testes frequentes, integração contínua e pequenas entregas.

Scrum: as funcionalidades poderiam ser organizadas em um Product Backlog e desenvolvidas durante sucessivas Sprints.

4.15 Qual é o Melhor Modelo?

Não existe uma resposta única. A escolha depende das características do projeto. Por exemplo:

- projetos com requisitos muito estáveis podem aceitar uma abordagem mais sequencial;
- projetos que precisam entregar funcionalidades progressivamente podem utilizar uma abordagem incremental;
- quando os requisitos ainda não estão claros, a prototipagem pode auxiliar;
- projetos com riscos elevados podem se beneficiar de uma abordagem orientada a riscos;
- projetos sujeitos a mudanças frequentes podem se beneficiar de práticas iterativas e ágeis.

Importante

O modelo de desenvolvimento deve ser escolhido de acordo com as **necessidades e características do projeto**, e não apenas por ser uma abordagem popular.

4.16 Resumo do Capítulo

Neste capítulo, estudamos diferentes maneiras de organizar o desenvolvimento de software.

O modelo **Cascata** organiza as atividades de maneira predominantemente sequencial.

O modelo **Incremental** constrói o software progressivamente por meio de incrementos.

O **RAD** enfatiza desenvolvimento rápido, iterações e participação do usuário.

A **Prototipagem** permite construir versões preliminares para ajudar na compreensão dos requisitos.

O modelo **Espiral** desenvolve o sistema em ciclos e utiliza a análise de riscos como elemento importante.

O **Processo Unificado** combina desenvolvimento iterativo e incremental em fases de Concepção, Elaboração, Construção e Transição.

A **Extreme Programming** enfatiza práticas técnicas, pequenas entregas, feedback e qualidade do código.

O **Scrum** organiza o trabalho em Sprints, nas quais a equipe produz incrementos, inspeciona os resultados e adapta o planejamento.

4.17 Fixando os Conceitos

1. O que é um modelo de ciclo de vida de software?
2. Qual é a principal característica do modelo Cascata?
3. Explique o significado de incremento no modelo Incremental.

4. Qual é a importância da participação do usuário no RAD?
5. O que é um protótipo?
6. Em que situação a prototipagem pode ser útil?
7. Qual é a principal característica do modelo Espiral?
8. Quais são as quatro fases do Processo Unificado?
9. Cite quatro práticas associadas à XP.
10. O que é uma Sprint?
11. Quais são as três responsabilidades que formam um Scrum Team?
12. Qual é a diferença principal entre XP e Scrum?
13. Existe um modelo de desenvolvimento adequado para todos os sistemas? Explique.

4.18 Atividade Prática

Considere que sua turma deverá desenvolver um **Sistema de Controle da Cantina Escolar**. O sistema deverá permitir:

- cadastrar produtos;
- controlar estoque;
- registrar vendas;
- consultar produtos;
- gerar relatórios.

Formem grupos e respondam:

1. Como esse sistema seria desenvolvido utilizando o modelo Cascata?
2. Divida o sistema em três possíveis incrementos.
3. Cite uma funcionalidade que poderia ser inicialmente desenvolvida como protótipo.
4. Identifique dois possíveis riscos que poderiam ser analisados em uma abordagem Espiral.
5. Distribua o desenvolvimento em quatro fases do Processo Unificado.
6. Escreva três possíveis histórias de usuário para uma equipe XP.
7. Crie um pequeno Product Backlog para o sistema.
8. Escolha três funcionalidades para uma primeira Sprint.
9. Entre os modelos estudados, qual seria utilizado pelo grupo? Justifique a escolha.

Capítulo 5

Introdução à UML

Nos capítulos anteriores, vimos que desenvolver um software envolve compreender um problema, identificar requisitos, analisar o sistema, projetar uma solução e posteriormente implementá-la.

Durante essas atividades, muitas informações precisam ser registradas e comunicadas entre as pessoas envolvidas no projeto. Uma das formas utilizadas para representar essas informações é a **UML – Unified Modeling Language**, em português, **Linguagem de Modelagem Unificada**.

A UML fornece recursos gráficos que permitem representar diferentes aspectos de um sistema antes ou durante sua construção. Neste capítulo, conheceremos os conceitos iniciais da UML. Os principais diagramas serão estudados detalhadamente nos próximos capítulos.

Objetivos de Aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- compreender o que é UML;
- identificar a finalidade da UML;
- reconhecer os elementos básicos utilizados na modelagem;
- distinguir diagramas estruturais e comportamentais;
- compreender por que um sistema pode possuir diferentes diagramas;
- reconhecer as principais visões utilizadas para representar um sistema;
- compreender a relação entre UML, análise e projeto de sistemas.

5.1 O que é UML?

A sigla UML vem do inglês *Unified Modeling Language*, que significa **Linguagem de Modelagem Unificada**.

A UML é uma linguagem visual utilizada para representar sistemas por meio de modelos e diagramas. Ela define símbolos e regras que permitem representar diferentes elementos de um sistema, como:

- usuários;
- funcionalidades;
- classes;
- objetos;
- mensagens;
- atividades;
- componentes;
- equipamentos onde o sistema será executado.

Conceito Central

A UML é uma linguagem visual utilizada para **modelar e representar diferentes aspectos de um sistema**.

Podemos representar essa ideia da seguinte forma:



Figura 5.1: A UML como instrumento para representar um sistema.

5.2 A UML não é uma Linguagem de Programação

O termo “linguagem” pode causar confusão. Java, Python e C++ são linguagens utilizadas para escrever programas que serão executados pelo computador. A UML possui outra finalidade. Ela é utilizada principalmente para **representar e comunicar informações sobre um sistema**. Observe a diferença:

Aspecto	UML	Linguagem de Programação
Finalidade	Modelar e representar o sistema	Implementar o software
Representação	Diagramas e elementos gráficos	Código-fonte
Exemplos	Diagrama de classes e casos de uso	Java, Python e C++
Resultado	Modelo do sistema	Programa executável

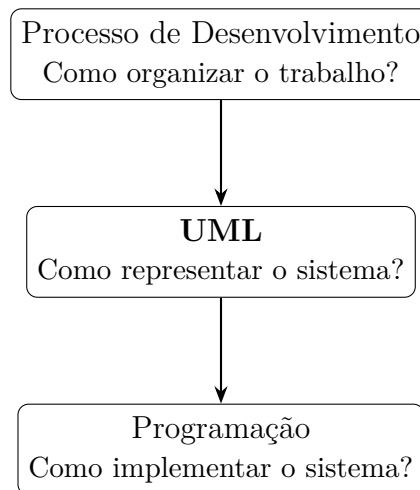
Importante

A UML **não é uma linguagem de programação**.

Ela auxilia na compreensão e representação do sistema que posteriormente poderá ser implementado utilizando uma linguagem de programação.

5.3 A UML também não é um Processo de Desenvolvimento

No capítulo anterior, estudamos Cascata, Incremental, Processo Unificado, XP e Scrum. Essas abordagens estão relacionadas à organização do **processo de desenvolvimento**. A UML possui uma finalidade diferente. Ela fornece uma linguagem para construir modelos. Assim, podemos utilizar UML em diferentes formas de desenvolvimento.



Resumo

Processo organiza o desenvolvimento.
UML representa o sistema.
Programação implementa o sistema.

5.4 Por que Utilizar UML?

Sistemas podem possuir muitas funcionalidades, classes, usuários e componentes. À medida que a complexidade aumenta, compreender o sistema apenas observando seu código-fonte torna-se difícil.

Os modelos ajudam a representar apenas os aspectos importantes para determinado objetivo. A utilização da UML pode auxiliar em tarefas como:

- compreender melhor o sistema;
- organizar informações;
- representar requisitos;
- representar a estrutura do software;
- representar o comportamento do sistema;
- facilitar a comunicação entre os participantes do projeto;
- documentar decisões tomadas durante o desenvolvimento;
- auxiliar atividades de análise e projeto.

5.4.1 Uma Analogia

Imagine que uma casa será construída. Antes da construção, podem ser produzidas diferentes plantas:

- planta baixa;
- planta elétrica;
- planta hidráulica;
- planta estrutural.

Todas representam a mesma casa, mas cada uma destaca um aspecto diferente. Em software ocorre algo semelhante.

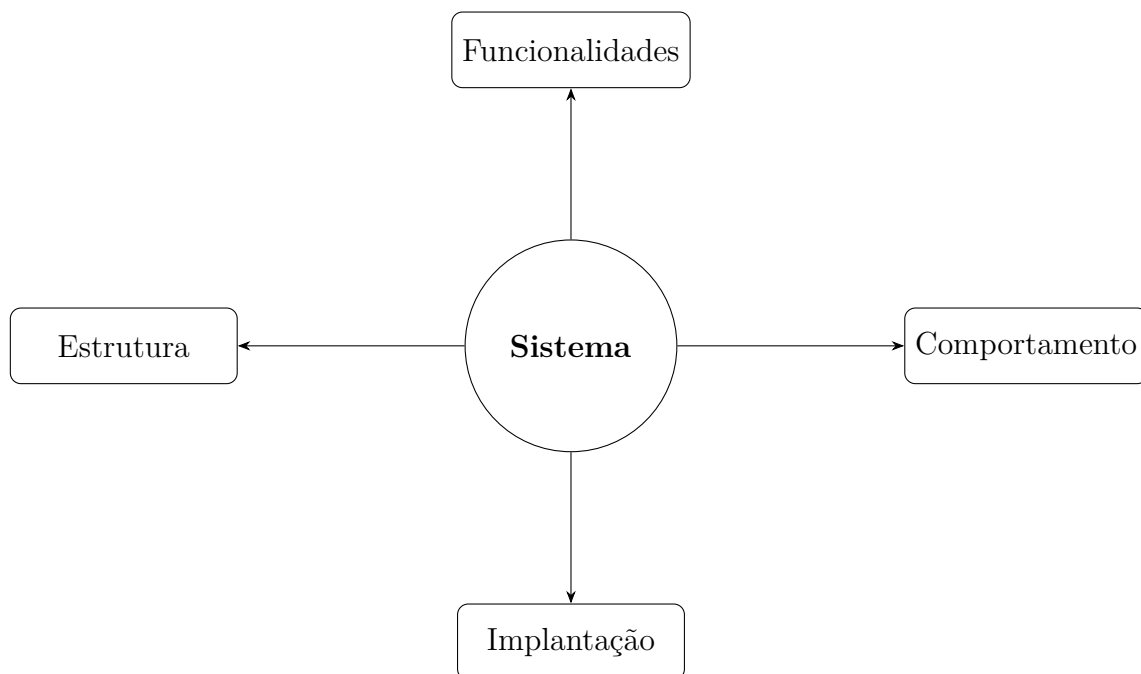


Figura 5.2: Um mesmo sistema pode ser representado sob diferentes aspectos.

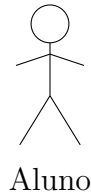
Por isso, a UML possui diferentes tipos de diagramas.

5.5 Elementos Básicos da UML

Os diagramas da UML são construídos utilizando diferentes elementos gráficos. Cada elemento possui um significado. Neste momento, não precisamos conhecer todos os símbolos da UML. Eles serão apresentados gradualmente nos próximos capítulos. Alguns exemplos iniciais são apresentados a seguir.

5.5.1 Ator

Um **ator** representa algo externo que interage com o sistema. Pode representar uma pessoa, outro sistema ou outro elemento externo.



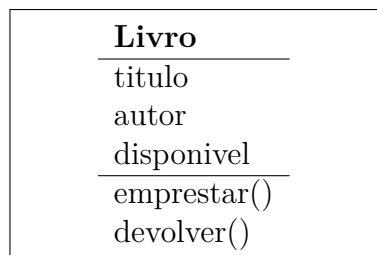
5.5.2 Caso de Uso

Um **caso de uso** representa uma funcionalidade oferecida pelo sistema.



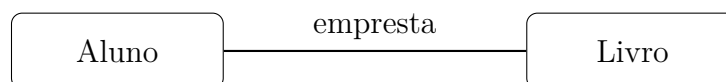
5.5.3 Classe

Uma **classe** representa um conjunto de objetos que possuem características e comportamentos semelhantes.



5.5.4 Relacionamentos

Elementos de um sistema normalmente não aparecem isolados. Eles podem possuir diferentes tipos de relacionamentos. De maneira simplificada:



Nos próximos capítulos aprenderemos que a UML possui diferentes tipos de relacionamentos e diferentes formas de representá-los.

5.5.5 Notas

A UML também permite adicionar informações textuais para explicar determinados elementos de um modelo. Essas informações podem ser representadas por **notas explicativas**.

O aluno pode possuir
até três empréstimos ativos.

Assim, um diagrama pode combinar elementos gráficos e informações textuais.

5.6 O que é um Diagrama?

Um **diagrama** é uma representação gráfica formada por elementos que possuem significados definidos. Cada diagrama procura mostrar determinado aspecto do sistema.

Conceito Central

Um **diagrama UML** representa uma determinada perspectiva ou aspecto de um sistema utilizando elementos gráficos padronizados.

Por exemplo:

- um diagrama pode mostrar **quem utiliza** o sistema;
- outro pode mostrar as **classes**;
- outro pode mostrar a **troca de mensagens**;
- outro pode mostrar a **sequência das atividades**;
- outro pode mostrar onde o sistema será **implantado**.

5.7 Tipos de Diagramas UML

Para facilitar o estudo, podemos inicialmente organizar os diagramas da UML em dois grandes grupos: (i) Diagramas Estruturais e (ii) Diagramas Comportamentais

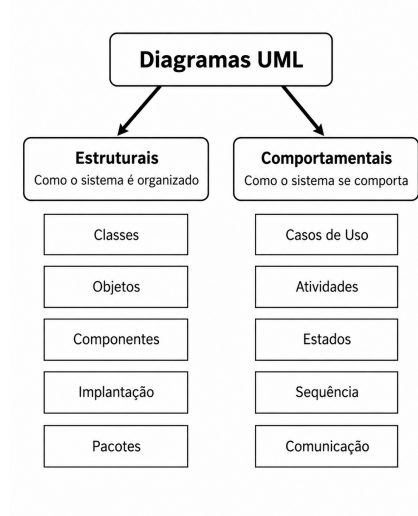


Figura 5.3: Classificação dos diagramas UML.

5.8 Diagramas Estruturais

Os **diagramas estruturais** representam aspectos relacionados à organização e à estrutura do sistema. Eles procuram responder perguntas como:

Quais elementos formam o sistema e como estão relacionados?

Entre os diagramas estruturais que estudaremos estão:

Diagrama de Classes: representa classes, características e relacionamentos.

Diagrama de Objetos: representa objetos e seus relacionamentos em determinado momento.

Diagrama de Componentes: representa componentes que formam o software.

Diagrama de Implantação: representa a distribuição do software nos equipamentos.

Diagrama de Pacotes: auxilia na organização dos elementos do sistema em grupos.

5.9 Diagramas Comportamentais

Os **diagramas comportamentais** representam aspectos relacionados ao funcionamento e às interações existentes no sistema. Eles procuram responder perguntas como:

O que acontece no sistema e como seus elementos interagem?

Entre os principais estão:

Diagrama de Casos de Uso: representa funcionalidades do sistema e atores externos.

Diagrama de Atividades: representa fluxos de atividades e decisões.

Diagrama de Estados: representa mudanças de estado de determinado elemento.

Diagrama de Sequência: representa a troca de mensagens entre objetos ao longo do tempo.

Diagrama de Comunicação: representa a comunicação existente entre objetos.

5.10 Diagramas de Interação

Alguns diagramas comportamentais possuem um foco especial na **interação entre os elementos do sistema**. Entre eles estão:

- diagrama de sequência;
- diagrama de comunicação.

Por exemplo, considere um aluno realizando um empréstimo.



Mais adiante, aprenderemos a representar formalmente essa interação utilizando um **Diagrama de Sequência**.

5.11 Precisamos Utilizar Todos os Diagramas?

Não. A UML fornece diferentes diagramas porque sistemas podem possuir diferentes níveis de complexidade e diferentes necessidades de documentação. Um sistema pequeno pode necessitar apenas de alguns modelos, enquanto um sistema maior pode exigir diferentes representações.

Importante

Não é necessário utilizar todos os diagramas UML em todos os projetos. Devemos selecionar os diagramas que ajudam a compreender e documentar os aspectos importantes do sistema.

Portanto:

Mais diagramas $\neq$ Melhor modelagem

Uma boa modelagem utiliza os diagramas necessários para compreender e comunicar adequadamente o sistema.

5.12 Visões de um Sistema

Além dos diferentes diagramas, um sistema também pode ser observado por diferentes **visões**. Uma visão destaca determinado conjunto de características do sistema. Entre as visões apresentadas na modelagem orientada a objetos estão:

- Visão de Casos de Uso;
- Visão de Projeto;
- Visão de Processo;
- Visão de Implementação;
- Visão de Implantação.

5.12.1 Visão de Casos de Uso

Apresenta o sistema do ponto de vista externo. Procura representar:

- quem utiliza o sistema;
- quais funcionalidades estão disponíveis;
- como os agentes externos interagem com o sistema.

5.12.2 Visão de Projeto

Representa aspectos relacionados à estrutura e ao comportamento da solução. Pode mostrar classes, objetos, interfaces e relacionamentos necessários para realizar as funcionalidades do sistema.

5.12.3 Visão de Processo

Representa aspectos relacionados à execução do software. Pode considerar características como:

- processos;
- concorrência;
- sincronização;
- desempenho.

5.12.4 Visão de Implementação

Representa a organização dos elementos utilizados para construir fisicamente o software. Pode envolver:

- componentes;
- módulos;
- bibliotecas;
- versões do software.

5.12.5 Visão de Implantação

Representa como o software será distribuído fisicamente. Pode representar elementos como:

- computadores;
- servidores;
- dispositivos;
- conexões entre equipamentos.

5.12.6 Representação das Visões

A Visão de Casos de Uso possui papel importante porque representa as funcionalidades percebidas pelos usuários e pode orientar a construção das demais visões.

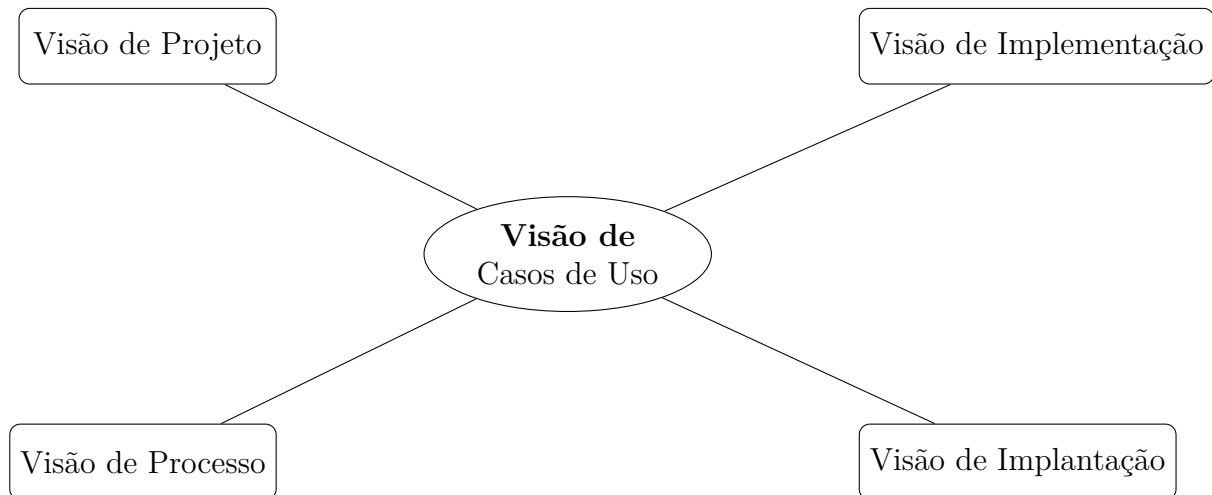


Figura 5.4: Diferentes visões utilizadas para representar um sistema.

5.13 UML e Análise de Sistemas

No Capítulo 2, vimos que a **análise** procura compreender:

O que o sistema deve fazer?

Durante a análise, a UML pode ser utilizada para construir modelos que representam o problema e os requisitos. Entre os diagramas que podem apoiar essa atividade estão:

- Diagrama de Casos de Uso;
- Diagrama de Classes;
- Diagrama de Atividades;
- Diagramas de Interação;
- Diagrama de Estados.

Por exemplo, podemos utilizar um Diagrama de Casos de Uso para mostrar que um aluno pode consultar livros e solicitar empréstimos.

5.14 UML e Projeto de Sistemas

Na atividade de **projeto**, procuramos responder:

Como o sistema será construído?

Os modelos criados durante a análise podem ser detalhados e novas informações relacionadas à solução tecnológica podem ser adicionadas. Durante o projeto, podem ser utilizados diagramas como:

- Diagrama de Classes de Projeto;

- Diagrama de Sequência;
- Diagrama de Comunicação;
- Diagrama de Componentes;
- Diagrama de Pacotes;
- Diagrama de Implantação.

5.15 Da Análise ao Projeto

Podemos representar de maneira simplificada a relação entre problema, análise, UML, projeto e implementação.



Figura 5.5: Relação simplificada entre análise, modelagem, projeto e implementação.

Essa representação é apenas didática. Na prática, análise e projeto podem ocorrer várias vezes durante o desenvolvimento, principalmente em processos iterativos e incrementais.

5.16 Exemplo Integrador: Biblioteca Escolar

Considere novamente o Sistema de Biblioteca Escolar. A UML pode representar diferentes aspectos desse sistema.

5.16.1 Funcionalidades

Um Diagrama de Casos de Uso poderá representar funcionalidades como:

- cadastrar livro;
- consultar livro;
- realizar empréstimo;
- registrar devolução.

5.16.2 Estrutura

Um Diagrama de Classes poderá representar elementos como:

- Aluno;
- Livro;
- Empréstimo;
- Bibliotecário.

5.16.3 Interação

Um Diagrama de Sequência poderá mostrar como os objetos colaboram durante um empréstimo:

Aluno → Sistema → Livro → Empréstimo

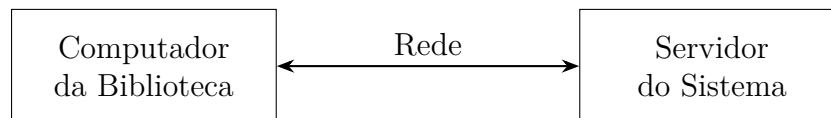
5.16.4 Atividades

Um Diagrama de Atividades poderá representar o fluxo:

Solicitar Livro → Verificar Disponibilidade → Registrar Empréstimo

5.16.5 Implantação

Um Diagrama de Implantação poderá representar onde o sistema será executado:



Observe que todos esses diagramas representam **o mesmo sistema**, mas destacam aspectos diferentes.

5.17 Resumo dos Principais Diagramas

Diagrama	O que representa?	Exemplo
Casos de Uso	Funcionalidades e interação com agentes externos	Aluno realiza empréstimo
Classes	Estrutura de classes e relacionamentos	Aluno, Livro e Empréstimo
Objetos	Objetos existentes em determinado momento	João e Livro001
Sequência	Troca de mensagens ao longo do tempo	Processo de empréstimo
Comunicação	Interação entre objetos	Objetos envolvidos no empréstimo
Atividades	Fluxo de atividades e decisões	Fluxo de devolução
Estados	Mudanças de estado	Disponível → Empréstado
Componentes	Organização dos componentes do software	Aplicação, serviço e banco de dados
Pacotes	Agrupamento de elementos do sistema	Cadastro, Empréstimos e Relatórios
Implantação	Distribuição física do software	Computador e servidor

5.18 Resumo do Capítulo

Neste capítulo, vimos que a UML é uma linguagem visual utilizada para modelar sistemas.

Ela não é uma linguagem de programação nem um processo de desenvolvimento.

A UML fornece elementos gráficos que podem ser combinados para construir diagramas.

Os diagramas permitem representar diferentes aspectos de um mesmo sistema e podem ser classificados, de maneira geral, em **estruturais** e **comportamentais**.

Os diagramas estruturais representam como o sistema está organizado, enquanto os comportamentais representam funcionalidades, atividades, estados e interações.

Também vimos que um sistema pode ser observado a partir de diferentes visões, como Casos de Uso, Projeto, Processo, Implementação e Implantação.

Durante a análise, os modelos ajudam a compreender **o que o sistema deve fazer**.

Durante o projeto, esses modelos podem ser refinados para definir **como o sistema será construído**.

5.19 Fixando os Conceitos

1. O que significa a sigla UML?
2. Explique com suas palavras o que é UML.
3. A UML é uma linguagem de programação? Explique.
4. A UML define um processo de desenvolvimento? Explique.
5. Cite três vantagens da utilização de modelos UML.
6. O que é um diagrama?
7. Qual é a principal diferença entre diagramas estruturais e comportamentais?
8. Cite três exemplos de diagramas estruturais.
9. Cite três exemplos de diagramas comportamentais.
10. Para que serve um Diagrama de Casos de Uso?
11. Para que serve um Diagrama de Classes?
12. Qual diagrama pode representar a troca de mensagens entre objetos ao longo do tempo?
13. Qual diagrama pode representar onde o software será executado?
14. É necessário utilizar todos os diagramas UML em todos os projetos? Explique.
15. Qual é a relação entre UML e análise de sistemas?
16. Qual é a relação entre UML e projeto de sistemas?

5.20 Atividade Prática

Considere novamente um **Sistema para a Cantina Escolar**. O sistema permite cadastrar produtos, registrar vendas, controlar estoque e consultar informações. Para cada situação abaixo, indique qual diagrama UML você utilizaria inicialmente e explique sua escolha.

1. Representar que o funcionário pode cadastrar produtos e registrar vendas.
2. Representar classes como Produto, Venda e Funcionário.
3. Representar a sequência de mensagens realizada durante uma venda.
4. Representar as etapas necessárias para concluir uma venda.
5. Representar a mudança de um produto de “Disponível” para “Sem Estoque”.
6. Representar os componentes principais da aplicação.
7. Representar em quais computadores ou servidores o sistema será executado.

Capítulo 6

Modelagem de Casos de Uso

No capítulo anterior, conhecemos a UML e vimos que diferentes diagramas podem representar diferentes aspectos de um sistema. Neste capítulo, estudaremos um dos modelos mais utilizados durante a análise de sistemas: o **Modelo de Casos de Uso**. Esse modelo ajuda a responder duas perguntas importantes:

Quem utiliza ou interage com o sistema?

Quais funcionalidades o sistema oferece a esses participantes?

A modelagem de casos de uso apresenta o sistema a partir de uma **textbfvisão** externa. Portanto, seu objetivo principal não é mostrar como o sistema funciona internamente, mas representar como os agentes externos interagem com suas funcionalidades **bezerra2015**.

Objetivos de Aprendizagem

Ao final deste capítulo, espera-se que você seja capaz de:

- compreender o que é um Modelo de Casos de Uso;
- diferenciar modelo e diagrama de casos de uso;
- identificar atores;
- identificar casos de uso;
- reconhecer atores primários e secundários;
- compreender os principais relacionamentos entre os elementos;
- construir um Diagrama de Casos de Uso;
- documentar atores;
- descrever casos de uso;
- identificar fluxo principal, fluxos alternativos e fluxos de exceção;
- reconhecer regras de negócio;
- relacionar requisitos de desempenho e de interface ao MCU;
- compreender o papel do MCU em processos iterativos;
- relacionar o MCU às atividades de análise, projeto e desenvolvimento.

6.1 Modelo de Casos de Uso

O **Modelo de Casos de Uso**, que podemos abreviar como **MCU**, representa as funcionalidades do sistema da maneira como são percebidas por seus agentes externos. Ele é construído principalmente a partir dos requisitos funcionais identificados durante o levantamento e a análise de requisitos.

Conceito Central

O **Modelo de Casos de Uso (MCU)** representa os possíveis usos de um sistema a partir do ponto de vista dos elementos externos que interagem com ele.

O MCU possui três elementos fundamentais:

- **atores;**
- **casos de uso;**
- **relacionamentos.**

Além disso, possui duas formas complementares de representação:

1. uma representação **gráfica**;
2. uma representação **textual**.

6.1.1 Parte Gráfica e Parte Textual

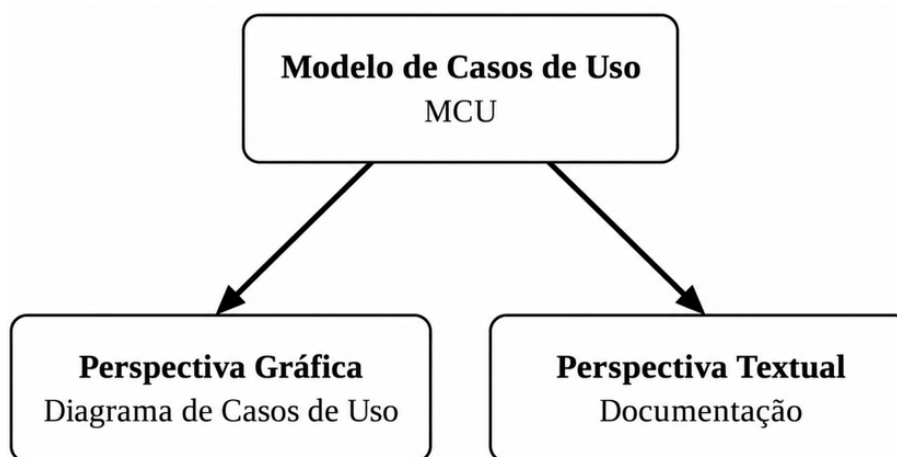


Figura 6.1: Classificação dos principais diagramas UML.

A parte gráfica fornece uma visão geral. A parte textual apresenta os detalhes necessários para compreender como cada funcionalidade deve se comportar.

Importante

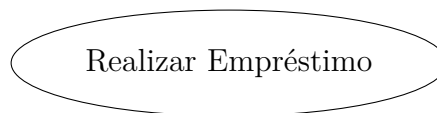
Modelo de Casos de Uso não é sinônimo de **Diagrama de Casos de Uso**. O diagrama é apenas uma das partes do modelo.

6.2 Caso de Uso

Um **caso de uso** representa uma funcionalidade oferecida pelo sistema a um ou mais agentes externos. Ele descreve uma sequência completa de interações necessária para alcançar determinado objetivo. Por exemplo, em uma biblioteca escolar:

- consultar livro;
- realizar empréstimo;
- registrar devolução;
- cadastrar livro.

Cada uma dessas ações pode representar um caso de uso.



6.2.1 Um Caso de Uso Representa um Objetivo

Um caso de uso não deve representar qualquer pequena ação realizada durante a utilização do sistema. Ele deve representar um objetivo completo do ator. Por exemplo:

Adequado	Pouco adequado
Realizar empréstimo	Digitar matrícula
Consultar livros	Clicar no botão Pesquisar
Registrar devolução	Informar código do livro
Cadastrar aluno	Preencher campo Nome

As ações da segunda coluna podem fazer parte de uma funcionalidade maior, mas normalmente não representam um objetivo completo.

Regra Prática

Um bom caso de uso deve representar algo que produza um **resultado de valor para um ator**.

6.2.2 Como Nomear um Caso de Uso

Uma forma simples é utilizar:

Verbo + Objeto

Exemplos:

- Cadastrar Livro;
- Consultar Livro;

- Realizar Empréstimo;
- Registrar Devolução;
- Emitir Relatório.

O nome deve permitir compreender rapidamente qual funcionalidade está sendo representada.

6.3 Atores

Um **ator** representa um papel desempenhado por um elemento externo que interage com o sistema. Um ator pode ser:

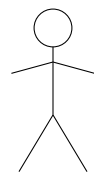
- uma pessoa;
- outro sistema;
- um equipamento;
- uma organização;
- outro elemento externo que troque informações com o sistema.

Conceito Central

Um **ator** representa um papel externo que interage com o sistema.

6.3.1 Representação de um Ator

Na UML, o ator humano é geralmente representado por uma figura semelhante a uma pessoa.



Aluno

6.3.2 Ator é um Papel

Um ator não representa necessariamente uma pessoa específica. Por exemplo: **João da Silva**, não seria normalmente utilizado como ator. O mais adequado seria: **Aluno**, porque diversos alunos podem desempenhar o mesmo papel em relação ao sistema. Da mesma forma, uma mesma pessoa pode desempenhar diferentes papéis.

Um professor pode, por exemplo, também atuar como coordenador.

Importante

O ator representa o papel desempenhado em relação ao sistema, e não necessariamente uma pessoa específica.

6.4 Atores Primários e Secundários

Um caso de uso pode envolver diferentes atores. Eles podem ser classificados em **primários** e **secundários**.

6.4.1 Ator Primário

O ator primário é aquele cujo objetivo é atendido pelo caso de uso. Normalmente ele inicia a interação com o sistema. Exemplo:

O **Aluno** solicita o empréstimo de um livro.

Nesse caso, o Aluno é o ator primário.

6.4.2 Ator Secundário

Um ator secundário participa da realização do caso de uso auxiliando o sistema ou o ator primário. Pode ser, por exemplo:

- outro sistema;
- um serviço externo;
- outro profissional;
- um dispositivo.

Em um sistema de pagamento, por exemplo, uma operadora de cartão pode participar como ator secundário.

Tipo	Papel
Ator primário	Busca alcançar um objetivo por meio do sistema.
Ator secundário	Auxilia na execução de uma funcionalidade.

6.5 Relacionamentos

Atores e casos de uso não aparecem isoladamente. A UML permite representar relacionamentos entre eles. Os principais relacionamentos utilizados no MCU são:

- comunicação;
- inclusão;
- extensão;
- generalização.

6.6 Relacionamento de Comunicação

O relacionamento mais simples ocorre entre um ator e um caso de uso. Ele indica que o ator participa daquela funcionalidade.

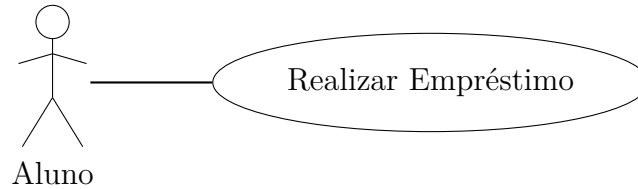


Figura 6.2: Relacionamento de comunicação entre ator e caso de uso.

A linha indica que o ator *Aluno* participa do caso de uso *Realizar Empréstimo*.

6.7 Relacionamento de Inclusão

O relacionamento de **inclusão** é utilizado quando um comportamento deve obrigatoriamente fazer parte de outro caso de uso. Na UML, utiliza-se:

«include»

Considere que todo empréstimo precisa verificar a disponibilidade do livro.

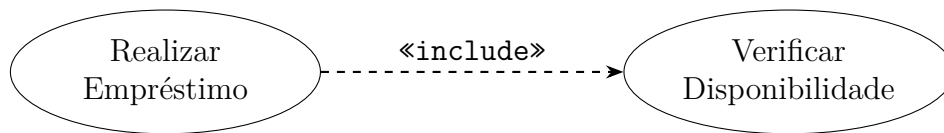


Figura 6.3: Exemplo de relacionamento de inclusão.

Nesse exemplo:

Sempre que o sistema realizar um empréstimo, deverá verificar a disponibilidade do livro.

Lembre-se

«include» representa um comportamento **necessário ou reutilizado obrigatoriamente**.

6.8 Relacionamento de Extensão

O relacionamento de **extensão** representa um comportamento adicional que ocorre somente em determinadas situações. Na UML, utiliza-se:

«extend»

Imagine que uma multa seja registrada somente quando um livro for devolvido com atraso.

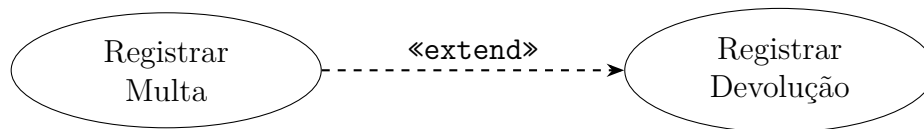


Figura 6.4: Exemplo de relacionamento de extensão.

O caso de uso *Registrar Devolução* pode ocorrer normalmente sem a execução de *Registrar Multa*. A multa somente será necessária quando determinada condição ocorrer.

Lembre-se

«extend» representa um comportamento **condicional ou eventual**.

6.9 Include ou Extend?

Uma forma simples de diferenciar os dois relacionamentos é perguntar:

	Include	Extend
Pergunta	Esse comportamento sempre precisa acontecer?	Esse comportamento acontece somente em determinada situação?
Execução	Obrigatória no contexto definido	Condicional ou eventual
Exemplo	Verificar disponibilidade ao emprestar um livro	Registrar multa quando houver atraso

6.10 Generalização

A generalização pode ser utilizada quando diferentes elementos possuem características semelhantes. Ela pode ocorrer:

- entre atores;
- entre casos de uso.

6.10.1 Generalização de Atores

Considere que alunos e professores possam utilizar funcionalidades comuns da biblioteca. Podemos criar um ator mais geral chamado *Usuário da Biblioteca*.

Aluno e Professor podem participar dos casos de uso associados ao ator mais geral, além de possuir funcionalidades específicas.

6.11 Diagrama de Casos de Uso

O **Diagrama de Casos de Uso**, ou DCU, representa graficamente:

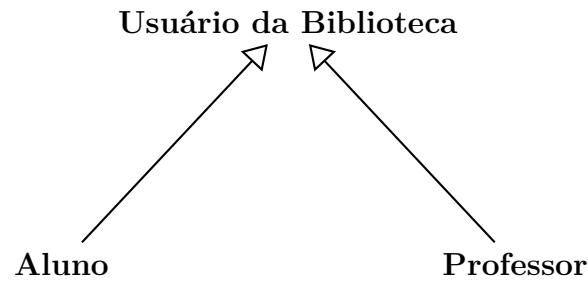


Figura 6.5: Representação simplificada da generalização de atores.

- os atores;
- os casos de uso;
- os relacionamentos.

Ele fornece uma visão geral das funcionalidades percebidas externamente.

Conceito Central

O Diagrama de Casos de Uso mostra **quem interage com quais funcionalidades do sistema**.

6.12 Fronteira do Sistema

É possível desenhar um retângulo representando a fronteira do sistema. Os casos de uso ficam **dentro** da fronteira. Os atores permanecem **fora** dela.



Figura 6.6: Exemplo simplificado de Diagrama de Casos de Uso.

6.13 Identificação dos Elementos do MCU

A construção do modelo começa pela identificação de seus elementos. Uma sequência didática possível é:

1. compreender a finalidade e a fronteira do sistema;
2. identificar os atores;
3. identificar os objetivos dos atores;
4. transformar esses objetivos em casos de uso;
5. identificar relacionamentos necessários;
6. construir o diagrama;
7. documentar atores e casos de uso;
8. validar o modelo com usuários e especialistas.

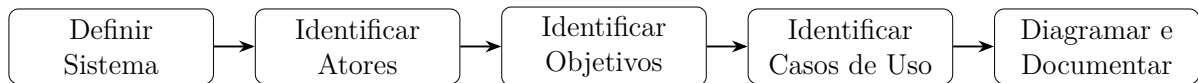


Figura 6.7: Sequência didática para identificação dos elementos do MCU.

6.14 Identificação dos Atores

Para identificar possíveis atores, algumas perguntas podem ajudar:

- Quem utiliza o sistema?
- Quem fornece informações ao sistema?
- Quem recebe informações do sistema?
- Que outros sistemas precisam se comunicar com ele?
- Existem equipamentos ou serviços externos envolvidos?

Considere novamente a biblioteca escolar. Podemos identificar:

- Aluno;
- Bibliotecário;
- Professor.

Se o software trocar informações com outro sistema, esse sistema externo também pode aparecer como ator.

6.14.1 Evite Confundir Ator com Parte do Sistema

Um ator deve estar fora da fronteira do sistema. Por exemplo:

Pode ser ator	Normalmente não é ator
Aluno	Tela de empréstimo
Bibliotecário	Banco de dados interno
Sistema Acadêmico externo	Classe Livro
Serviço externo	Módulo de cadastro interno

6.15 Identificação dos Casos de Uso

Depois de identificar os atores, devemos procurar compreender **quais objetivos eles desejam alcançar utilizando o sistema**. Para cada ator, podemos perguntar:

- O que ele precisa fazer utilizando o sistema?
- Que resultado espera obter?
- Que informações precisa consultar?
- Que informações precisa registrar ou atualizar?

Para o Bibliotecário, por exemplo:

- Cadastrar Livro;
- Realizar Empréstimo;
- Registrar Devolução;
- Consultar Empréstimos.

6.15.1 Casos de Uso Primários

Os casos de uso primários estão diretamente associados aos principais objetivos e processos do domínio. Por exemplo:

- Realizar Empréstimo;
- Registrar Devolução;
- Consultar Acervo.

6.15.2 Casos de Uso Secundários

Outras funcionalidades podem ser necessárias para apoiar o funcionamento do sistema. Por exemplo:

- manter dados cadastrais;
- gerenciar usuários;
- sincronizar informações com sistemas externos.

Durante a análise inicial, é importante não perder de vista as funcionalidades que realmente produzem valor para os usuários **bezerra2015**.

6.16 Construção do Modelo de Casos de Uso

Depois de identificar atores e funcionalidades, podemos construir o MCU. Lembre-se de que esse trabalho envolve duas perspectivas.

Perspectiva	Conteúdo
Gráfica	Atores, casos de uso e relacionamentos representados no DCU.
Textual	Descrição dos atores, casos de uso, fluxos e informações complementares.

6.17 Construção do Diagrama de Casos de Uso

Um procedimento simples para construir o DCU é:

1. desenhar a fronteira do sistema;
2. posicionar os casos de uso dentro dela;
3. posicionar os atores fora;
4. associar cada ator aos casos de uso dos quais participa;
5. adicionar **include**, **extend** ou generalização somente quando necessário;
6. revisar a legibilidade do diagrama.

Boa Prática

O objetivo de um diagrama é **comunicar**.

Se houver tantos elementos e relacionamentos que o diagrama se torne difícil de compreender, é melhor dividi-lo em mais de um diagrama.

Bezerra destaca que, em sistemas maiores, diferentes DCUs podem ser criados para apresentar grupos relacionados de casos de uso **bezerra2015**.

6.18 Documentação dos Atores

A documentação de um ator pode ser bastante simples. Devemos registrar:

- nome;
- descrição do papel desempenhado.

Exemplo:

Ator	Descrição
Aluno	Usuário da biblioteca que consulta o acervo e solicita empréstimos.
Bibliotecário	Responsável pelo cadastro do acervo e pelo registro de empréstimos e devoluções.
Professor	Usuário da biblioteca que consulta e realiza empréstimos de materiais.

6.19 Documentação dos Casos de Uso

O diagrama fornece apenas uma visão geral. Para compreender detalhadamente uma funcionalidade, podemos criar uma **descrição textual do caso de uso**.

A UML não obriga a utilização de um único formato de documentação. Cada equipe pode escolher os itens adequados às necessidades do projeto **bezerra2015**. Uma descrição mais completa pode apresentar:

- nome;
- identificador;
- importância;
- sumário;
- ator primário;
- atores secundários;
- condições;
- fluxo principal;
- fluxos alternativos;
- fluxos de exceção;
- pós-condições;
- regras de negócio;
- histórico;
- notas adicionais.

6.20 Fluxo Principal

O **fluxo principal** descreve a sequência normal de interação entre o ator e o sistema. Exemplo:

Caso de Uso: Realizar Empréstimo

1. O Aluno solicita o empréstimo de um livro.
2. O sistema identifica o aluno.
3. O sistema verifica a disponibilidade do livro.
4. O sistema verifica se o aluno pode realizar empréstimos.
5. O sistema registra o empréstimo.
6. O sistema informa a data prevista para devolução.

O fluxo deve utilizar linguagem clara e próxima do domínio do usuário.

6.21 Fluxos Alternativos

Um **fluxo alternativo** representa outra forma válida de realizar o caso de uso. Por exemplo:

Fluxo Alternativo: Renovar empréstimo

Durante o empréstimo, o aluno solicita uma renovação. O sistema verifica se a renovação é permitida e registra a nova data de devolução.

O objetivo ainda pode ser alcançado, mas por um caminho diferente.

6.22 Fluxos de Exceção

Um **fluxo de exceção** representa uma situação de erro ou condição que impede o fluxo normal de continuar como previsto. Exemplo:

Fluxo de Exceção: Livro indisponível

No passo 3, o sistema identifica que o livro já está emprestado.

O sistema informa que o livro não está disponível.

O empréstimo não é registrado.

6.23 Pré-condições e Pós-condições

6.23.1 Precondição

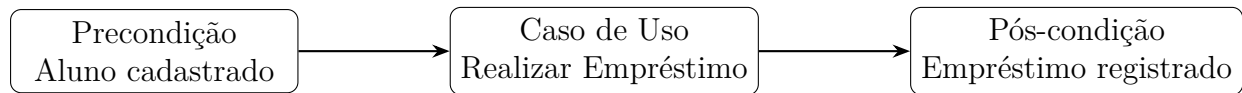
Uma precondição informa o que precisa ser verdadeiro antes da execução do caso de uso. Exemplo:

O aluno deve estar cadastrado na biblioteca.

6.23.2 Pós-condição

Uma pós-condição indica o estado alcançado depois que o caso de uso é realizado. Exemplo:

O empréstimo foi registrado e o livro passou a constar como emprestado.



6.24 Exemplo Completo de Documentação

A tabela a seguir apresenta uma versão adequada para esta apostila.

Tabela 6.1: Descrição do caso de uso Realizar Empréstimo

Identificador	CSU01
Nome	Realizar Empréstimo
Sumário	Registrar o empréstimo de um livro para um usuário da biblioteca.
Ator primário	Aluno
Ator secundário	Bibliotecário
Precondição	O aluno deve estar cadastrado e habilitado para realizar empréstimos.
Fluxo principal	1. O aluno solicita o empréstimo. 2. O sistema identifica o aluno. 3. O sistema identifica o livro. 4. O sistema verifica a disponibilidade. 5. O sistema verifica as regras de empréstimo. 6. O sistema registra o empréstimo. 7. O sistema informa a data de devolução.
Fluxo alternativo	O aluno pode selecionar outro exemplar disponível do mesmo título.
Fluxo de exceção	Se o livro estiver indisponível, o sistema informa a situação e o empréstimo não é registrado.
Pós-condição	O empréstimo foi registrado e o exemplar passou a constar como emprestado.
Regras de negócio	RN01 e RN02.

6.25 Evite Detalhes de Interface na Análise

Na descrição inicial de um caso de uso, é importante concentrar-se no **que o ator deseja realizar**, e não nos detalhes da tela. Por exemplo:

Mais adequado na análise	Detalhe desnecessário neste momento
O aluno solicita o empréstimo.	O aluno clica no botão verde “Emprestar”.
O usuário informa sua matrícula.	O usuário digita a matrícula na caixa localizada no canto superior.
O bibliotecário confirma a devolução.	O bibliotecário pressiona o botão Confirmar.

Isso mantém o caso de uso independente da tecnologia ou do desenho específico da interface. Os detalhes da interface poderão ser definidos posteriormente durante o projeto **bezerra2015**.

6.26 Documentação Suplementar ao MCU

O Modelo de Casos de Uso concentra-se principalmente nos **requisitos funcionais**. Entretanto, um sistema possui outras informações importantes. Entre elas:

- regras de negócio;
- requisitos de desempenho;
- requisitos de interface gráfica.

Essas informações podem ser documentadas separadamente e associadas aos casos de uso correspondentes.

6.27 Regras de Negócio

Uma **regra de negócio** representa uma política, condição ou restrição existente no domínio onde o sistema será utilizado. Ela não é simplesmente uma decisão de programação.

É uma regra que existe porque a organização funciona dessa maneira. Exemplos para uma biblioteca escolar:

- um aluno pode possuir no máximo três empréstimos ativos;
- o prazo normal de empréstimo é de sete dias;
- um aluno com empréstimo atrasado não pode realizar novo empréstimo;
- livros de consulta local não podem sair da biblioteca.

Uma mesma regra pode influenciar vários casos de uso. Por isso, é útil atribuir um identificador como:

RN01, RN02, RN03, ...

Tabela 6.2: Exemplo de documentação de regras de negócio

Código	Nome	Descrição
RN01	Limite de empréstimos	Um aluno pode possuir no máximo três empréstimos ativos.
RN02	Prazo de devolução	O prazo padrão para devolução é de sete dias.
RN03	Pendência	Aluno com livro atrasado não pode realizar novo empréstimo.

6.28 Requisitos de Desempenho

Um requisito de desempenho estabelece condições relacionadas ao funcionamento esperado do sistema. Pode definir, por exemplo:

- tempo máximo de resposta;
- quantidade de operações;
- volume de dados;
- número esperado de usuários.

Exemplo:

Tabela 6.3: Exemplos de requisitos de desempenho associados a casos de uso

Caso	Funcionalidade	Desempenho esperado
CSU01	Realizar Empréstimo	Resposta em até 2 segundos.
CSU02	Consultar Livro	Resposta em até 2 segundos.
CSU03	Gerar Relatório	Resposta em até 10 segundos.

Esses requisitos não precisam aparecer dentro do desenho do DCU. Eles podem ser registrados em documentação complementar.

6.29 Requisitos de Interface Gráfica

Também podem existir requisitos relacionados à interface gráfica. Exemplos:

- a interface deve ser adequada aos computadores disponíveis na biblioteca;
- as informações devem possuir boa legibilidade;
- campos obrigatórios devem ser claramente identificados;
- o sistema deve manter um padrão visual entre suas telas.

Esses requisitos podem ser associados aos casos de uso correspondentes.

Atenção

Não confunda **requisito de interface** com a descrição detalhada de como cada tela será implementada. Na análise, o caso de uso deve permanecer concentrado no objetivo e nas interações essenciais.

6.30 MCU em um Processo de Desenvolvimento Iterativo

Como vimos no Capítulo 4, um sistema pode ser desenvolvido em várias iterações. O MCU pode ajudar a organizar esse desenvolvimento. Os casos de uso podem ser distribuídos entre diferentes iterações. Por exemplo:

Iteração	Casos de Uso
Iteração 1	Cadastrar Livro e Consultar Livro
Iteração 2	Realizar Empréstimo e Registrar Devolução
Iteração 3	Gerar Relatórios e Consultar Histórico

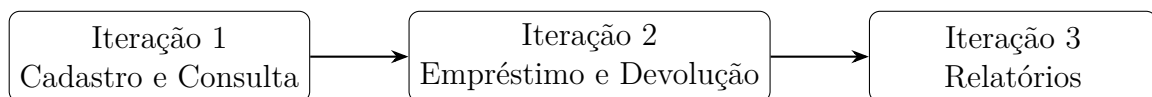


Figura 6.8: Casos de uso distribuídos em iterações.

Casos de uso de maior importância ou maior risco podem receber prioridade nas primeiras iterações **bezerra2015**.

6.31 O MCU nas Atividades de Análise e Projeto

Durante a **análise**, procuramos compreender:

O que o sistema deve fazer?

Nesse momento, o MCU ajuda a detalhar os requisitos funcionais. As descrições devem utilizar principalmente a linguagem do usuário e evitar decisões tecnológicas. Durante o **projeto**, procuramos definir:

Como o sistema será construído?

Os casos de uso passam a orientar modelos mais detalhados relacionados à solução.



Na análise, portanto, seria adequado escrever:

O usuário solicita a consulta.

e não:

O usuário dá duplo clique no botão Consulta.

A segunda descrição já está vinculada a uma solução de interface específica.

6.32 O MCU nas Demais Atividades do Desenvolvimento

O MCU não termina sua utilidade depois do levantamento dos requisitos. Ele pode orientar diferentes atividades posteriores.

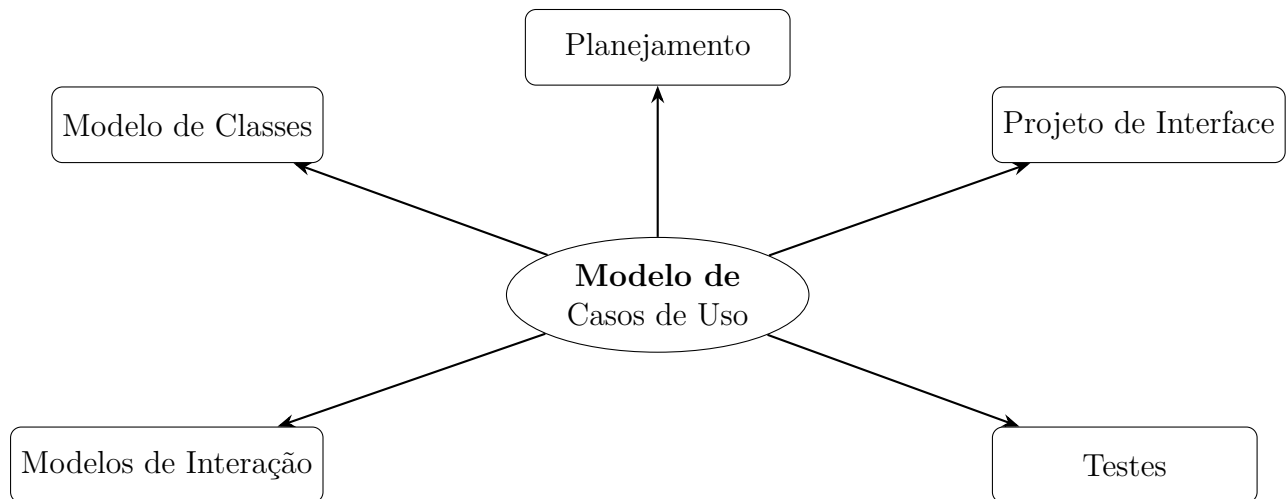


Figura 6.9: Relação do Modelo de Casos de Uso com outras atividades do desenvolvimento.

6.32.1 Planejamento

Os casos de uso podem ajudar a:

- definir prioridades;
- organizar iterações;
- acompanhar funcionalidades desenvolvidas.

6.32.2 Modelagem de Classes

As descrições dos casos de uso podem auxiliar na identificação das classes necessárias para realizar as funcionalidades do sistema. Por exemplo, ao estudar *Realizar Empréstimo*, podem surgir conceitos como:

- Aluno;
- Livro;
- Empréstimo.

Essa relação será estudada no próximo capítulo.

6.32.3 Modelagem de Interações

Os cenários descritos nos casos de uso podem ser utilizados posteriormente para estudar como os objetos colaboram. Essa colaboração poderá ser representada por diagramas como:

- Diagrama de Sequência;
- Diagrama de Comunicação.

6.32.4 Projeto de Interface

Os casos de uso ajudam a identificar quais interações deverão ser possibilitadas pela interface. Entretanto, os detalhes de apresentação são definidos principalmente durante o projeto.

6.32.5 Testes

Os fluxos descritos nos casos de uso também fornecem cenários úteis para verificar o comportamento esperado do sistema. Por exemplo, o caso de uso *Realizar Empréstimo* sugere verificar:

- empréstimo realizado normalmente;
- livro indisponível;
- aluno com pendência;
- aluno com limite de empréstimos atingido.

6.33 Exemplo Integrador: Sistema de Biblioteca

Vamos reunir os elementos identificados neste capítulo.

6.33.1 Atores

- Aluno;
- Professor;
- Bibliotecário.

6.33.2 Casos de Uso

- Consultar Livro;
- Realizar Empréstimo;
- Registrar Devolução;
- Cadastrar Livro;

- Consultar Empréstimos;
- Gerar Relatório.

6.33.3 Regras de Negócio

- RN01 – aluno pode possuir no máximo três empréstimos;
- RN02 – prazo normal de empréstimo é de sete dias;
- RN03 – aluno com pendência não pode realizar novo empréstimo.

6.33.4 Modelo Simplificado

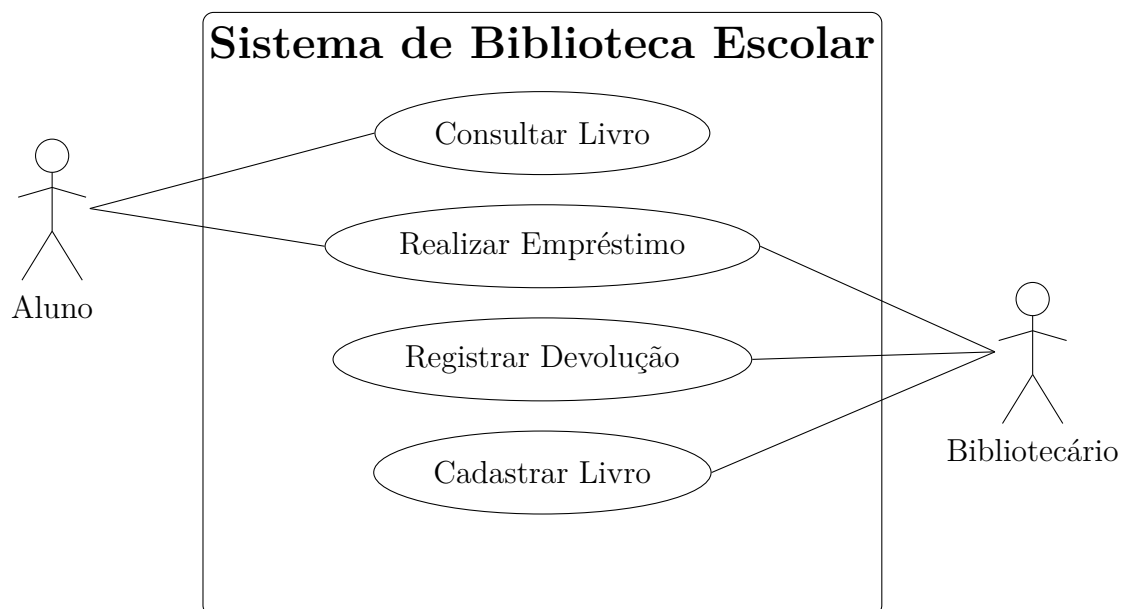


Figura 6.10: Modelo simplificado de casos de uso da biblioteca escolar.

6.34 Erros Comuns na Modelagem de Casos de Uso

Ao construir os primeiros modelos, alguns erros são frequentes.

6.34.1 Representar pequenas ações como casos de uso

Evite:

- Digitar CPF;
- Clicar em Confirmar;
- Selecionar Livro.

Procure identificar o objetivo completo.

6.34.2 Colocar elementos internos como atores

Banco de dados, classes e componentes internos normalmente não são atores.

6.34.3 Confundir MCU com Fluxograma

O DCU não mostra a ordem de execução das funcionalidades. Ele mostra principalmente:

Atores + Funcionalidades + Relacionamentos

6.34.4 Utilizar relacionamentos em excesso

Muitos relacionamentos **include**, **extend** e generalizações podem tornar o modelo difícil de compreender. A clareza deve ser prioridade **bezerra2015**.

6.34.5 Desenhar e não documentar

O diagrama sozinho normalmente não apresenta detalhes suficientes para compreender todos os comportamentos do sistema. Por isso:

Diagrama + Descrição Textual = MCU mais completo

6.35 Resumo do Capítulo

Neste capítulo, vimos que o Modelo de Casos de Uso representa o sistema a partir de uma perspectiva externa. Seus principais elementos são atores, casos de uso e relacionamentos.

Um ator representa um papel externo que interage com o sistema.

Um caso de uso representa uma sequência de interações associada a um objetivo do ator.

O Diagrama de Casos de Uso apresenta graficamente esses elementos, enquanto a documentação textual fornece detalhes sobre seu funcionamento.

Os principais relacionamentos estudados foram comunicação, inclusão, extensão e generalização.

Também vimos que uma descrição de caso de uso pode conter condições, fluxo principal, fluxos alternativos, fluxos de exceção e pós-condições.

Regras de negócio, requisitos de desempenho e requisitos de interface podem complementar as informações fornecidas pelo MCU.

Finalmente, vimos que os casos de uso podem orientar diferentes atividades do desenvolvimento, como planejamento, modelagem de classes, modelagem de interações, projeto de interface e testes.

6.36 Fixando os Conceitos

1. O que é um Modelo de Casos de Uso?
2. Qual é a diferença entre MCU e DCU?

3. O que é um ator?
4. Um ator precisa necessariamente ser uma pessoa? Explique.
5. Qual é a diferença entre ator primário e ator secundário?
6. O que é um caso de uso?
7. Por que “Digitar senha” normalmente não representa um bom caso de uso?
8. Cite quatro exemplos de nomes adequados para casos de uso.
9. Para que serve o relacionamento de comunicação?
10. Explique o relacionamento «include».
11. Explique o relacionamento «extend».
12. Qual é a diferença entre inclusão e extensão?
13. O que é generalização entre atores?
14. O que é fluxo principal?
15. O que é fluxo alternativo?
16. O que é fluxo de exceção?
17. Explique a diferença entre precondição e pós-condição.
18. O que é uma regra de negócio?
19. Cite dois exemplos de requisitos de desempenho.
20. Por que os casos de uso escritos durante a análise devem evitar detalhes da interface gráfica?
21. Como o MCU pode ser utilizado em um processo iterativo?

6.37 Atividade Prática

Considere um **Sistema para a Cantina Escolar**.

O sistema deverá permitir que funcionários controlem os produtos, registrem vendas e acompanhem o estoque. Realize as seguintes atividades:

1. Identifique pelo menos três atores.
2. Identifique pelo menos seis casos de uso.
3. Classifique os atores em primários ou secundários quando aplicável.
4. Crie um Diagrama de Casos de Uso.
5. Identifique uma possível relação de inclusão.

6. Identifique uma possível relação de extensão.
7. Escreva duas regras de negócio.
8. Selecione um caso de uso e descreva:
 - (a) nome;
 - (b) identificador;
 - (c) sumário;
 - (d) ator primário;
 - (e) precondição;
 - (f) fluxo principal;
 - (g) fluxo alternativo;
 - (h) fluxo de exceção;
 - (i) pós-condição.
9. Defina um requisito de desempenho relacionado a esse caso de uso.
10. Defina um requisito de interface gráfica relacionado à funcionalidade.